

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

Направление	01.03.02 Прикладная математика и информатика
Профиль	Математическое обеспечение программно-информационных систем
Факультет	КТИ
Кафедра	МО ЭВМ

К защите допустить

Зав. кафедрой

А.А. Лисс

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

**Тема: Программный модуль для повторной идентификации объектов
в видеопотоке с камер видеонаблюдения**

Студент			Р.П. Дамакин
		<i>подпись</i>	
Руководитель	к.т.н., доцент		В.В. Романцев
		<i>подпись</i>	
Консультанты			В.К. Клюканов
		<i>подпись</i>	
	к.т.н., доцент		М.М. Заславский
		<i>подпись</i>	
	к.э.н., доцент		Л.Н. Ковалик
		<i>подпись</i>	

Санкт-Петербург

2026

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Утверждаю

Зав. кафедрой МО ЭВМ

_____ А.А. Лисс

«___» _____ 20__ г.

Студент Дамакин Р.П. Группа 2384

Тема работы: Программный модуль для повторной идентификации объектов в видеопотоке с камер видеонаблюдения

Место выполнения ВКР: Санкт-Петербургский государственный электротехнический университет «ЛЭТИ» им. В.И.Ульянова (Ленина).

Исходные данные (технические требования): видеопоследовательности MOT17 и тестовые видеофрагменты; язык программирования Python; библиотеки OpenCV, Ultralytics YOLO, Torchreid, PyTorch; ОС Windows 10/11; вычислительная система с RTX 3050 Laptop.

Содержание ВКР:

Анализ предметной области, обзор аналогов, составление требований к системе, проектирование программного модуля, разработка программного модуля, проведение экспериментов

Перечень отчетных материалов: пояснительная записка, иллюстративный материал

Дополнительные разделы: Экономическое обоснование ВКР.

Дата выдачи задания
«06» апреля 2026 г.

Дата представления ВКР к защите
«11» июня 2026 г.

Студент	_____	Р.П. Дамакин
Руководитель	к.т.н., доцент _____	В.В. Романцев
Консультант	_____	В.К. Клюканов

КАЛЕНДАРНЫЙ ПЛАН ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Утверждаю

Зав. кафедрой МО ЭВМ

_____ А.А. Лисс

« 20 Г.

Студент Дамакин Р.П.

Группа 2384

Тема работы: Программный модуль для повторной идентификации объектов в видеопотоке с камер видеонаблюдения

№ п/п	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	06.04 – 10.04
2	Обзор предметной области и существующих аналогов	10.04 – 17.04
3	Формулировка требований к решению и постановка задачи	17.04 – 22.04
4	Описание метода решения и проектирование архитектуры	22.04 – 27.04
5	Реализация программного модуля	13.04 – 30.04
6	Исследование свойств разработанного решения	30.04 – 07.05
7	Экономическое обоснование проекта	07.05 – 11.05
8	Оформление пояснительной записки	06.04 – 14.05
9	Оформление иллюстративного материала	11.05 – 14.05
7	Предзащита	28.05 – 05.06

Студент

Р.П. Дамакин

Руководитель

К.Т.Н., ДОЦЕНТ

В.В. Романцев

Консультант

В.К. Ключанов

РЕФЕРАТ

Пояснительная записка 85 стр., 13 рис., 12 табл., 50 ист.

ПОВТОРНАЯ ИДЕНТИФИКАЦИЯ, ВИДЕОПОТОК, ВИДЕОНАБЛЮДЕНИЕ, ВИДЕОАНАЛИТИКА, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, ОБНАРУЖЕНИЕ ОБЪЕКТОВ, ЛОКАЛЬНЫЙ ТРЕКИНГ, ГАЛЕРЕЯ ИДЕНТИЧНОСТЕЙ, ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ, YOLO, TORCHREID, ПРОГРАММНЫЙ МОДУЛЬ.

Объектом исследования является процесс повторной идентификации объектов в видеопотоке с камер видеонаблюдения.

Предметом исследования является устойчивость повторного назначения глобального идентификатора объекту после разрыва наблюдения в видеопотоке.

Цель работы: программный модуль повторной идентификации объектов в видеопотоке с камер видеонаблюдения, обеспечивающий обнаружение объектов, извлечение признаков представлений, сопоставление с галереей идентичностей и повторное назначение глобального идентификатора после разрыва наблюдения.

В процессе работы был проведен анализ предметной области повторной идентификации объектов в видеопотоках камер видеонаблюдения, рассмотрены существующие подходы, библиотеки и прикладные решения. На основе результатов анализа были сформулированы требования к разрабатываемому программному модулю, предложена его модульная архитектура и реализованы основные подсистемы: обнаружение объектов, локальный трекинг, извлечение признаков и галерея идентичностей. Были проведены экспериментальная проверка работоспособности системы, параметрический анализ и экономическое обоснование проекта.

Результатом работы стал программный модуль повторной идентификации объектов, обеспечивающий долгосрочное хранение идентичностей, повторное связывание объектов после разрыва наблюдения и возможность конфигурационного управления параметрами работы [1].

ABSTRACT

In the course of the work, an analysis of the subject area of object re-identification in video streams from surveillance cameras was carried out, and existing approaches, libraries, and applied solutions were reviewed. Based on the results of the analysis, requirements for the software module under development were formulated, its modular architecture was proposed, and the main subsystems were implemented: object detection, local tracking, feature extraction, and an identity gallery. Experimental validation of the system, parametric analysis, and economic justification of the project were also performed.

The result of the work is a software module for object re-identification that provides long-term identity storage, repeated association of objects after observation interruptions, and configurable control of operating parameters [1].

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	8
ВВЕДЕНИЕ.....	9
1 Анализ предметной области	12
1.1 Задача повторной идентификации объектов.....	12
1.2 Формальная постановка задачи повторной идентификации	14
1.3 Архитектурные подходы к построению систем ReID.....	15
1.4 Обзор существующих аналогов и решений	17
1.4.1 Краткая характеристика аналогов	18
1.4.2 Критерии сравнения аналогов	21
1.4.3 Выводы по результатам сравнения	23
2 Проектирование программного модуля повторной идентификации	25
2.1 Постановка задачи.....	25
2.2 Требования к разрабатываемой системе.....	25
2.3 Описание архитектуры программного модуля	27
2.3.1 Система обнаружения объектов	30
2.3.2 Система локального трекинга.....	31
2.3.3 Подсистема извлечения признаков	32
2.3.4 Галерея идентичностей и механизм сопоставления	34
2.4 Выводы по выбору решения	36
3 Реализация программного модуля.....	38
3.1 Организация программного модуля.....	38
3.2 Архитектура программной реализации	40
3.3 Описание параметров конфигурации.....	42
3.4 Сборка и сценарий использования проекта конечным пользователем	44
3.5 Выводы по результатам разработки.....	46
4 Исследование свойств решения.....	48
4.1 Экспериментальные данные и вычислительные условия.....	48
4.2 Проверка функциональных возможностей	49
4.3 Метрики экспериментальной оценки.....	54

4.4 Экспериментальная оценка устойчивости повторного назначения идентификатора	58
4.5 Результаты экспериментальной оценки разработанного программного модуля.....	64
4.6 Сравнение решения с аналогами	67
4.7 Выводы по результатам исследования.....	68
5 Экономическое обоснование проекта	70
5.1 Календарный план выполнения.....	70
5.2 Расходы на оплату труда	71
5.4 Нематериальные расходы.....	75
5.5 Амортизационные отчисления	75
5.6 Затраты на содержание и эксплуатацию.....	76
5.7 Услуги сторонних организаций	77
5.8 Накладные расходы.....	77
5.9 Вывод.....	77
ЗАКЛЮЧЕНИЕ	79
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	81

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Повторная идентификация объектов (ReID) – задача сопоставления наблюдений одного и того же объекта, полученных в разные моменты времени, в том числе после его исчезновения и повторного появления в кадре;

Локальный трекинг – краткосрочное связывание обнаружений объекта на соседних кадрах в единую траекторию;

Глобальная идентичность – долговременно хранимое представление объекта в системе, используемое для повторного присвоения идентификатора;

Галерея идентичностей – структура данных, содержащая глобальные идентификаторы, их признаковые прототипы и служебные метаданные;

Признаковое представление объекта – числовой вектор, описывающий визуальные характеристики объекта и используемый для сопоставления;

Область интереса – фрагмент изображения, содержащий обнаруженный объект и передаваемый на последующие этапы обработки;

Локальная траектория – последовательность наблюдений объекта, поддерживаемая трекером в пределах ограниченного временного интервала;

Открытый мир – сценарий работы системы, при котором состав наблюдаемых объектов заранее неизвестен и может изменяться во времени;

IoU – Intersection over Union, мера пересечения по объединению;

α – коэффициент сглаживания при обновлении прототипа;

FPS – скорость обработки кадров;

SOTA – state of the art;

SDK – Software Development Kit.

ВВЕДЕНИЕ

В современных системах видеонаблюдения непрерывно формируются большие объемы видеоданных, ручной анализ которых становится трудоемким, дорогостоящим и плохо масштабируемым. По этой причине задачи автоматизированной видеоаналитики приобретают все большее практическое значение в сфере безопасности, транспортного мониторинга, контроля доступа и анализа поведения объектов в наблюдаемой сцене. В отраслевых обзорах рынка видеоаналитики отдельно подчеркиваются устойчивый рост данного направления, расширение числа прикладных сценариев и возрастающая роль интеллектуальной обработки видеопотоков [2,3]. Однако для многих прикладных сценариев недостаточно только обнаружить объект в отдельном кадре или сопровождать его на коротком временном интервале. В реальных условиях требуется сохранять согласованную идентичность объекта при его временном исчезновении, повторном появлении, а в более сложной постановке и при переходе между различными камерами. Эту задачу решает повторная идентификация объектов. Она дополняет подсистемы детекции и локального трекинга. Обнаружение объектов определяет местоположение объекта в кадре, а трекинг связывает соседние наблюдения в локальную траекторию, однако оба подхода ограничены текущей сценой, краткосрочным временным окном и качеством геометрической ассоциации. При плотном потоке объектов, перекрытиях, изменении ракурса, освещения и масштаба, а также при длительных разрывах наблюдения этих средств недостаточно для устойчивого восстановления идентичности. В связи с этим возникает необходимость в программном модуле, который использует признаковое представление объекта, хранит накопленные сведения о ранее наблюдавшихся идентичностях и может повторно назначать глобальный идентификатор после разрыва локальной траектории [4,5].

Актуальность темы определяется тем, что существующие решения либо встраивают повторную идентификацию внутрь алгоритма трекинга и ориентируются прежде всего на ведение траектории в пределах короткого времени, либо представлены в виде исследовательских инструментов и соревновательных платформ, недостаточно приспособленных к прямой интеграции в прикладные системы видеонаблюдения. Особенно заметен этот разрыв в сценариях открытого мира, где требуется различать ранее известных и новых объектов в длительном видеопотоке и поддерживать идентичность вне рамок одной локальной траектории [4,6]. Поэтому практический интерес представляет разработка воспроизводимого и конфигурируемого программного модуля, который можно встроить в потоковый конвейер обработки видео, независимо изменяя его составные части: детекцию, локальное сопровождение, извлечение признаков и долгосрочную галерею идентичностей.

Объектом исследования является процесс повторной идентификации объектов в видеопотоке с камер видеонаблюдения.

Предмет исследования – устойчивость повторного назначения глобального идентификатора объекту после разрыва наблюдения в видеопотоке.

Цель работы – это программный модуль повторной идентификации объектов в видеопотоке с камер видеонаблюдения, обеспечивающий обнаружение объектов, извлечение признаковых представлений, сопоставление с галереей идентичностей и повторное назначение глобального идентификатора после разрыва наблюдения.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Выполнить анализ предметной области повторной идентификации объектов и рассмотреть существующие архитектурные подходы, библиотеки и прикладные решения.

2. Сформулировать требования к программному модулю, ориентированному на потоковую обработку видеоданных и долгосрочное хранение идентичностей.
3. Спроектировать модульную архитектуру, разделяющую подсистемы обнаружения объектов, локального трекинга, извлечения признаков, галереи идентичностей и регистрации результатов.
4. Реализовать программный модуль повторной идентификации с конфигурационным управлением параметрами запуска, поддержкой сохранения состояния галереи и возможностью подключения различных моделей.
5. Провести экспериментальную проверку работоспособности системы и исследовать влияние параметров детекции, сопоставления и обновления галереи на качество повторной идентификации и производительность.
6. Выполнить экономическое обоснование проекта.

Практическая значимость заключается в том, что разработанный модуль может использоваться как встраиваемый компонент систем видеоаналитики для сохранения и повторного назначения идентификаторов объектов после разрыва наблюдения. Модуль поддерживает конфигурационное управление, работу с видеопотоком, сохранение состояния галереи и формирование результатов обработки, что упрощает его интеграцию в прикладные системы видеонаблюдения.

Структура работы включает анализ предметной области, проектирование программного модуля, описание реализации, экспериментальную проверку, экономическое обоснование проекта и формулирование направлений дальнейшего развития системы.

1 Анализ предметной области

1.1 Задача повторной идентификации объектов

Повторная идентификация объектов представляет собой задачу сопоставления наблюдений одного и того же объекта, полученных в разные моменты времени и, зачастую, разными камерами с непересекающимися полями зрения [4]. В научной и прикладной литературе наиболее развиты направления повторной идентификации людей и транспортных средств, так как они напрямую связаны с задачами интеллектуального видеонаблюдения, транспортной аналитики и обеспечения безопасности [4,7]. В отечественных работах данное направление также рассматривается прежде всего применительно к системам видеонаблюдения и задаче устойчивого сопоставления людей по изображениям камер [5,8]. В рамках данной работы используется общий термин "объект", однако базовым сценарием тестирования текущей реализации программного модуля является повторная идентификация человека.

Задача повторной идентификации принципиально отличается от задач обнаружения объектов и трекинга. Подсистема обнаружения локализует объект в кадре, но не определяет, наблюдался ли он ранее. Алгоритм трекинга связывает результаты обнаружения на соседних кадрах в единую траекторию, однако в типичном случае делает это в пределах локального временного окна, опираясь на геометрию движения и кратковременную устойчивость внешнего вида [9,10]. Повторная идентификация, напротив, ориентирована на поиск устойчивого признакового представления объекта, позволяющего распознавать его после исчезновения из кадра, при повторном появлении и, в более сложной постановке задачи, при переходе между разными камерами [4]. Практическая сложность задачи определяется рядом факторов: изменением ракурса и масштаба объекта, частичными окклюзиями, фоновым шумом, нестабильным освещением, схожестью внешнего вида разных объектов одного класса и разрывами во времени между наблюдениями [4,11]. Для систем круглосуточного видеонаблюдения

к указанным факторам добавляется смена режима съемки. Для камер видимого диапазона ночной переход к инфракрасному режиму приводит к потере цветовой информации и формирует отдельную задачу кросс-модальной повторной идентификации [11,12]. В работах по сопоставлению видимого и инфракрасного диапазонов показано, что именно для 24-часовых систем видеонаблюдения требуется отдельное изучение такого режима, так как большинство камер в темное время суток автоматически переключаются в инфракрасный режим [12].

Современная теория повторной идентификации также различает исследовательские сценарии закрытого мира и более прикладные сценарии открытого мира [4]. В первом случае предполагается, что множество идентичностей контролируемо и известно в рамках эксперимента на наборе данных. Во втором случае система должна работать в реальной среде, где состав объектов заранее неизвестен, а сами идентичности появляются и исчезают асинхронно. Практическая значимость такого сценария подтверждается, в частности, исследованиями для розничной среды, где требуется различать постоянных и эпизодических посетителей по длительным видеозаписям и данным камер с верхним ракурсом [6]. Для разрабатываемого программного модуля существенен именно сценарий открытого мира, так как система должна не только ранжировать пары изображений в рамках тестового протокола, но и поддерживать непрерывную работу на реальном видеопотоке.

В прикладной системе видеонаблюдения пользователем такого решения является как оператор, наблюдающий за видеопотоком, так и разработчик или инженер по интеграции, подключающий модуль повторной идентификации к существующему конвейеру видеоаналитики. Результат работы должен иметь форму программного модуля, который принимает данные от подсистемы получения видеопотока, использует детекцию и локальное сопровождение объектов, образует галерею глобальных идентификаторов и служебную информацию для дальнейшей обработки.

Такой модуль может применяться при анализе архивных видеозаписей, при потоковой обработке данных с камер и при исследовании влияния параметров повторной идентификации на качество работы системы.

1.2 Формальная постановка задачи повторной идентификации

Пусть входной видеопоток представлен последовательностью кадров (1):

$$V = \{F_1, F_2, \dots, F_T\}, \quad (1)$$

где V – входной видеопоток; F_t – кадр с номером t ; T – число кадров в видеопоследовательности.

На каждом кадре F_t подсистема обнаружения формирует множество наблюдений (2):

$$D_t = \{d_1^t, d_2^t, \dots, d_{n_t}^t\}, \quad (2)$$

где D_t – множество обнаруженных объектов на кадре t ; d_i^t – i -е наблюдение на кадре t ; n_t – число наблюдений на данном кадре. Каждое наблюдение d_i^t содержит ограничивающий прямоугольник, метку класса и оценку уверенности. Для каждого наблюдения формируется признаковый вектор e_i^t , описывающий внешний вид объекта в признаковом пространстве модели повторной идентификации.

Требуется определить отображение текущего наблюдения в множество глобальных идентичностей (3):

$$f(d_i^t) \rightarrow g_j, \quad (3)$$

где g_j – существующая идентичность из галереи. Если сходство текущего признакового вектора с сохраненными прототипами ниже заданного порога, система должна создать новую идентичность g_{new} .

В такой постановке задача не сводится к сопровождению объекта на соседних кадрах. Необходимо поддерживать долговременную галерею идентичностей, которая позволяет повторно назначать объекту ранее созданный глобальный идентификатор после разрыва наблюдения, удаления локальной траектории или перезапуска программы.

1.3 Архитектурные подходы к построению систем ReID

С точки зрения временной глубины сопоставления целесообразно выделять краткосрочную и долгосрочную повторную идентификацию. Краткосрочный подход характерен для MOT-систем (multi-object tracking), где признаки объекта используются для поддержания его идентификатора на соседних кадрах и для преодоления коротких окклюзий. Такой подход реализуется, например, в Deep SORT [9], где визуальный дескриптор обучается заранее, а затем применяется как метрика ассоциации в потоковом режиме. Аналогичную постановку развивают ByteTrack [10] и BoT-SORT [13], ориентированные на сохранение цельной траектории и уменьшение числа переключений идентификаторов в пределах одного видеоролика. Долгосрочная архитектура повторной идентификации строится иначе: она предполагает наличие внешней галереи признаков или прототипов идентичностей, которая живет дольше отдельной локальной траектории и может использоваться при повторном появлении объекта спустя значительный промежуток времени. В такой архитектуре также может применяться локальный трекинг; в таком случае он отвечает за временную связность наблюдений, а модуль повторной идентификации решает задачу повторного присвоения глобального идентификатора при потере объекта. Именно этот подход ближе к реальным системам видеонаблюдения, где объект может надолго исчезать из поля зрения камеры, возвращаться позже или наблюдаться после перезапуска системы.

По числу камер различаются однокамерные и многокамерные сценарии. Однокамерная повторная идентификация ограничивается одним источником видеосигнала и фактически решает задачу долгосрочного сопоставления внутри одной оптической сцены. При этом не исключено использование однокамерной повторной идентификации в системах видеонаблюдения с подвижной камерой или изменяющейся сценой. Многокамерная повторная идентификация значительно сложнее, поскольку требует инвариантности к межкамерным различиям: углу обзора,

экспозиции, фокусному расстоянию, положению объекта в кадре и временному разрыву между его появлениями [4,7]. В CityFlow прямо указывается, что многокамерный трекинг в городской среде фактически складывается из трех взаимосвязанных задач: однокамерного трекинга, повторной идентификации по изображению и учета пространственно-временной информации [7].

Дополнительно архитектуры повторной идентификации можно разделить по способу интеграции с остальным конвейером обработки видео. Первый вариант заключается во встраивании повторной идентификации внутрь алгоритма трекинга. Именно так устроено большинство популярных МОТ-решений: признаки используются как вспомогательный механизм ассоциации обнаружений [9,10,13,14]. Это удобно для быстрого запуска и хорошо работает на непрерывных видеосценах, но плохо подходит для случаев, когда требуется восстановить личность после длительного пропадания объекта или перенести идентификатор объекта между сессиями. Второй вариант основан на модульной архитектуре, где обнаружение объектов, краткосрочный трекинг, извлечение признаков и галерея идентичностей оформлены как отдельные подсистемы. Такой подход дает возможность независимо заменять модуль обнаружения, модуль извлечения признаков и стратегию сопоставления, а также сохранять галерею во внешнем состоянии. Для прикладного программного модуля этот путь представляется более рациональным, поскольку повышает воспроизводимость, переносимость и удобство интеграции в уже существующие программные комплексы.

С точки зрения моделей признаков в литературе можно выделить два крупных класса. Первый класс составляют компактные СНС-архитектуры (сверточные нейронные сети), ориентированные на практичность и высокую скорость вывода. Характерным примером является модель OSNet, построенная как легковесная сеть разномасштабных признаков и предназначенная специально для задач повторной идентификации людей

[15]. В отечественных исследованиях этому направлению соответствуют и подходы на основе нейросетевых составных дескрипторов, ориентированные на повышение устойчивости сопоставления изображений из систем видеонаблюдения [8]. Второй класс составляют архитектуры на основе механизма внимания, в которых усилен учет глобального контекста и зависимостей между фрагментами изображения. Примером использования такого подхода в задаче повторной идентификации может служить проект TransReID [16], представленный на конференции ICCV 2021. Дополнительное развитие этого направления демонстрируют методы, усиливающие глобальные признаки через механизм внимания и угловую оптимизацию функции потерь, что позволяет повышать различимость признаков представлений [17]. Подходы на основе механизма внимания демонстрируют высокое качество на тестовых наборах данных, но на данный момент редко применяются в системах видеоаналитики в реальном времени из-за высокой вычислительной стоимости.

1.4 Обзор существующих аналогов и решений

Для формирования обзора существующих аналогов поиск проводился по трем практическим сценариям. Первый сценарий был направлен на выявление готовых MOT-решений, в которых повторная идентификация используется как механизм ассоциации обнаружений. Для этого анализировались Google Scholar, arXiv, официальная документация Ultralytics и репозитории GitHub по запросам: «multi object tracking», «person re-identification tracking», «multi object tracking re-identification», «open world person re-identification», «reid tracking github», «Deep SORT ReID», «BoT-SORT ReID», «Ultralytics tracking ReID», «повторная идентификация объектов видеонаблюдение», «трекинг». Второй сценарий касался промышленных и инфраструктурных платформ, пригодных для интеграции в видеоконвейер реального времени. Поиск выполнялся по официальной документации NVIDIA DeepStream, OpenVINO, а также материалам AI City Challenge и CityFlow с использованием запросов, связанных с повторным

сопоставлением целей, многокамерным трекингом, долгосрочной идентификацией и галереями признаков: «multi-camera tracking», «long-term re-identification gallery», «re-id demo», «OpenVINO reidentification», «NVIDIA reidentification». Третий сценарий был ориентирован на поиск библиотек и архитектур, пригодных для построения собственного модуля извлечения признаков и сопоставления идентичностей. В этом случае использовались Google Scholar, eLibrary, CyberLeninka, IEEE Xplore, MDPI, arXiv, GitHub и публикации конференций CVPR/ICCV по следующим запросам в поисковых системах: «open source person re-identification», «re-id tool», «re-id python library», «модуль повторной идентификации», «Torchreid», «FastReID», «Open ReID». При отборе решений учитывались наличие открытой публикации или официальной документации, применимость к задачам видеоаналитики реального времени, наличие компонента повторной идентификации или выделяемого модуля извлечения признаков, возможность интеграции в прикладной проект на Python, а также наличие явных ограничений, существенных для сценария долговременной идентификации.

1.4.1 Краткая характеристика аналогов

Наиболее массово повторная идентификация представлена как часть MOT-систем. Классическим примером является Deep SORT, где внешний визуальный дескриптор уменьшает число переключений идентификаторов и повышает устойчивость траекторий при окклюзиях [9]. Логика решения при этом остается ориентированной на ведение локальной траектории: идентификатор поддерживается по мере движения объекта в ролике, а не как долговременно хранимая сущность. Дальнейшее развитие получили ByteTrack и BoT-SORT. ByteTrack позиционируется авторами как простое, быстрое и сильное решение для трекинга и улучшает качество за счет ассоциации всех обнаружений, даже низкого качества [10]. BoT-SORT дополняет это использование компенсации движения камеры и основанной на внешнем виде объекта ассоциации, достигая высоких значений метрик на

тестовом наборе MOTChallenge [13]. Однако и ByteTrack, и BoT-SORT по исходной постановке ориентированы прежде всего на задачу ведения траекторий в пределах видеопоследовательности. Следовательно, их применение для долгосрочной идентификации в системе видеонаблюдения требует дополнительного внешнего слоя хранения и сопоставления глобальных идентичностей, а также отдельной системы обработки видеопотока в реальном времени.

Схожая ситуация наблюдается в экосистеме Ultralytics: официальная документация описывает режим трекинга как расширение обнаружения объектов, поддерживающее BoT-SORT и ByteTrack. Система повторной идентификации в проекте рассматривается как опциональный механизм улучшения ассоциации при окклюзиях [14]. Параметры настройки BoT-SORT позволяют указать количество кадров, в течение которых должны сохраняться признаки идентифицированного объекта, но авторы прямо указывают на направленность использования повторной идентификации как метода повышения устойчивости траекторий. Следовательно, такая система не подходит при использовании в сценариях с возможной долгосрочной потерей объекта из кадра.

Более широкие промышленные решения представлены платформами NVIDIA DeepStream и OpenVINO. DeepStream содержит библиотеку NvMultiObjectTracker и поддерживает ускоренное извлечение признаков повторной идентификации за счет использования технологии TensorRT [18]. Вместе с тем документация фиксирует важное ограничение: если объект покидает кадр или оказывается скрытым дольше, чем допускает параметр, ограничивающий время жизни признаков, при возвращении ему присваивается новый идентификатор. Система повторной идентификации в таком случае подчинена логике долгоживущего трекера, но не заменяет самостоятельную галерею идентичностей. Кроме того, технологический набор DeepStream привязан к инфраструктуре NVIDIA, TensorRT и GStreamer, что ускоряет систему в случае использования мощного

оборудования от NVIDIA, но делает такой подход менее универсальным и сложным для встраивания в произвольные проекты видеоаналитики. Проекты, построенные на основе инструментария OpenVINO обычно имеют возможность использования долгосрочной галереи признаков, работающей также в многокамерных системах [19,20]. Преимущество OpenVINO заключается в наличии готовых оптимизированных моделей и демонстрационных приложений для повторной идентификации человека и транспорта. Ограничениями являются привязанность к оборудованию Intel, ограниченность доступных нейросетевых моделей, а также сложность тонкой настройки конфигурации.

Отдельную группу решений составляют соревновательные и исследовательские решения для многокамерного видеонаблюдения. Так, в работе CityFlow многокамерный трекинг и повторная идентификация рассматриваются как часть городской аналитики на масштабном наборе данных с синхронизированными видеопотоками, калибровочной информацией и разметкой [7]. В AI City Challenge 2024 отдельно выделялось направление многокамерного трекинга людей; масштаб сгенерированных данных был увеличен примерно до 1300 камер и 3400 отслеживаемых людей, а в оценивание кандидатов был введен бонус за поддержку потокового трекинга [21]. Такие проекты являются источниками масштабных тестовых наборов и современных, так называемых state-of-the-art (SOTA) методов для развития области и создания новых подходов, но по своей природе они ориентированы на соревнование, разметку, оценочные протоколы и калибровку сцены, и не подходят для создания внедряемого в системы наблюдения в реальном времени программного модуля.

Для задач исследования собственных решений повторной идентификации существуют открытые библиотеки, ориентированные на обучение, тестирование и интеграцию моделей повторной идентификации в прикладные проекты. Среди таких библиотек можно выделить Torchreid [22], FastReID [23] и Open-ReID [24]. Их использование удобно при модульной

разработке, поскольку они предоставляют унифицированные средства работы с наборами данных, моделями и метриками качества, а также допускают загрузку предобученных и собственных весов. В частности, TorchreID содержит готовые инструменты для обучения и тестирования моделей повторной идентификации, а также механизм извлечения признаков, позволяющий встроить логику формирования признаков идентифицируемых объектов в собственные программные решения. FastReID, в свою очередь, представляет собой модульный инструментарий для задач общей повторной идентификации и поддерживает конфигурируемое обучение, оценку и развертывание моделей. Open-ReID ориентирована на исследовательское применение и предоставляет единый интерфейс для работы с различными наборами данных, моделями и метриками оценки. Ограничением подобных библиотек остается конечный перечень поддерживаемых нейросетевых архитектур, однако возможность дообучения и подключения собственных весов делает их удобной основой для исследования и построения систем повторной идентификации объектов.

1.4.2 Критерии сравнения аналогов

Сравнение аналогов выполняется по четырем критериям:

Горизонт идентификации характеризует срок сохранения идентичности объекта. Для критерия определяются значения:

- ЛТ – локальная траектория: идентичность поддерживается во время активного сопровождения объекта.
- БТ – буфер трека: идентичность сохраняется при кратковременной потере объекта в пределах настроенного буфера.
- СГ – сеансовая галерея: идентичность хранится в отдельной галерее в течение сеанса работы.

Поддерживаемые объекты характеризуют классы, доступные для ReID без переработки архитектуры. Значения критерия:

- Человек – решение ориентировано на повторную идентификацию людей.
- Человек/транспорт – доступны сценарии или модели для людей и транспортных средств.
- Классы детектора – состав объектов определяется моделью обнаружения.
- Настраиваемые классы – расширение возможно через пользовательские веса детектора и ReID-модели.

Форма решения характеризует способ внедрения аналога:

- Трекер – готовый алгоритм сопровождения.
- Фреймворк – промышленный или демонстрационный конвейер видеоаналитики.
- Библиотека ReID – средство обучения, оценки или извлечения признаков без полного видеоконвейера.

Настройка параметров характеризует изменяемые элементы решения:

- Трекинг – настраиваются параметры сопровождения и ассоциации.
- Модель – доступны выбор модели, весов и метрик ReID.
- Конвейер – настраиваются параметры нескольких этапов обработки видео.

Рассмотренные решения сведены в таблицу 1.

Таблица 1 – Сравнение аналогов

Аналог	Горизонт идентификации	Поддерживаемые объекты	Форма решения	Настройка параметров
Deer SORT [9]	ЛТ	Человек	Трекер	Трекинг
ByteTrack [10]	ЛТ	Классы детектора	Трекер	Трекинг

Продолжение таблицы 1

Аналог	Горизонт идентификации	Поддерживаемые объекты	Форма решения	Настройка параметров
BoT-SORT [13]	БТ	Человек	Трекер	Трекинг
Ultralytics Tracking [14]	БТ	Классы детектора	Трекер / API	Конвейер
NVIDIA DeepStream [18]	БТ	Человек/транспорт	Фреймворк	Конвейер
OpenVINO ReID / deep-object-reid [19, 20]	СГ	Человек/транспорт	Фреймворк	Конвейер
Torchreid [22]	Не задается	Настраиваемые классы	Библиотека ReID	Модель
FastReID [23]	Не задается	Настраиваемые классы	Библиотека ReID	Модель

1.4.3 Выводы по результатам сравнения

Сравнение показало, что рассмотренные аналоги закрывают разные части задачи, но не совпадают с постановкой данной работы полностью. Deep SORT, ByteTrack, BoT-SORT и Ultralytics Tracking ориентированы прежде всего на сопровождение объектов внутри текущей видеопоследовательности. В таких решениях признаки внешнего вида используются как часть механизма трекинга, поэтому для долгосрочного повторного назначения идентификатора после разрыва наблюдения требуется дополнительный слой хранения и сопоставления.

Torchreid и FastReID предоставляют средства работы с ReID-моделями. Они позволяют обучать, оценивать и подключать модели извлечения

признаков. Эти решения не являются готовыми видеоконвейерами. Для их применения нужно отдельно реализовать получение кадров, детекцию, локальное сопровождение, галерею, сохранение состояния и интерфейс запуска. При этом такие библиотеки полезны для расширения состава объектов, так как допускают подключение собственных весов.

NVIDIA DeepStream и OpenVINO предлагают более законченные конвейеры видеоаналитики, однако они сильнее связаны с собственными технологическими экосистемами. Это снижает удобство свободного изменения логики галереи, способа сопоставления и состава экспериментальных параметров.

На основании сравнения можно выделить нишу разрабатываемого решения: программный модуль, в котором потоковая обработка видео совмещается с отделенной от локального трекинга галереей идентичностей, сохранением состояния между сеансами и конфигурационным управлением параметрами. Для расширения количества обнаруживаемых классов важна также возможность использовать собственные веса моделей обработки. Качество ReID для новых классов при этом зависит от выбранной модели признаков. Следующие разделы будут сосредоточены на обосновании выбранной архитектуры, критериев конфигурирования и особенностей реализации долгосрочной идентификации в сценарии реального времени.

2 Проектирование программного модуля повторной идентификации

2.1 Постановка задачи

Необходимо разработать программный модуль повторной идентификации объектов в видеопотоке, предназначенный для включения в конвейер видеоаналитики. Модуль должен принимать на вход видеофайл, потоковый источник или адрес камеры, выполнять обнаружение объектов заданных классов, связывать соседние наблюдения в локальные траектории, формировать признаковые представления объектов и сопоставлять их с долговременно хранимой галереей идентичностей.

Сформулированная постановка задачи определяет требования к разрабатываемому модулю. Поскольку система должна работать с видеопотоком, а не только с отдельными изображениями, в требования включается поддержка файловых и потоковых источников. Так как рассмотренные трекеры в основном сохраняют идентичность в пределах локальной траектории или короткого буфера, для разрабатываемого решения требуется отдельная галерея глобальных идентификаторов. Необходимость экспериментальной проверки также требует вынесения параметров обработки в конфигурацию и сохранения результатов работы в виде журналов, метрик, размеченного видео и состояния галереи.

2.2 Требования к разрабатываемой системе

На основе результатов анализа предметной области были сформулированы требования к разрабатываемому программному модулю повторной идентификации.

С функциональной точки зрения программный продукт должен обеспечивать следующие возможности:

1. Принимать видеоданные из файлового источника и из потокового источника, используемого в системах видеонаблюдения.

2. Выполнять обнаружение объектов целевого класса на входных кадрах и передавать найденные области интереса на последующие этапы обработки.
3. Поддерживать локальный трекинг объектов внутри текущей видеопоследовательности, чтобы связывать наблюдения соседних кадров в краткосрочные траектории.
4. Формировать признаковое представление каждого найденного объекта и использовать его для сопоставления с ранее накопленными представлениями.
5. Создавать глобальный идентификатор для нового объекта, если совпадение в галерее не найдено.
6. Повторно назначать существующий глобальный идентификатор объекту при его повторном появлении после разрыва локальной траектории или временного исчезновения из кадра.
7. Сохранять и загружать состояние галереи идентичностей между сеансами работы, чтобы ранее созданные идентификаторы могли использоваться после перезапуска программы.
8. Ограничивать размер галереи и учитывать актуальность хранимых записей, чтобы рост числа идентичностей не приводил к неконтролируемому увеличению потребления памяти.
9. Формировать структурированные результаты работы: размеченный видеоряд, журнал выполнения, файл метрик и сохраненное состояние галереи.
10. Обеспечивать воспроизводимый запуск за счет вынесения изменяемых параметров, путей к моделям, источников видеоданных и режимов сохранения в конфигурацию.

С точки зрения интеграции программный продукт должен обладать следующими свойствами:

1. Входной интерфейс модуля должен быть формализован и не зависеть от конкретного способа запуска системы. Это означает, что

модуль должен принимать стандартизированное описание источника видеоданных и набора параметров обработки.

2. Внешнее взаимодействие с модулем должно осуществляться через конфигурацию и документированные параметры запуска, что позволяет включать его в более крупные конвейеры обработки без переработки внутренней логики.
3. Результаты работы модуля должны быть представлены в унифицированном виде и включать как визуализированные данные, так и метрики или структурированное состояние, пригодное для последующего анализа другими подсистемами.
4. Программный продукт не должен быть жестко связан с конкретным интерфейсом пользователя. Это позволяет использовать его как в составе настольного приложения, так и в составе серверного или иного внешнего решения.
5. Модуль должен быть конфигурируемым, чтобы одна и та же архитектура могла применяться в различных сценариях эксплуатации без изменения исходного кода.
6. Программный продукт должен сопровождаться инструкцией по запуску и описанием входных и выходных данных, поскольку без этого его встраивание в сторонние проекты будет затруднено даже при корректной реализации алгоритма.

2.3 Описание архитектуры программного модуля

Общая структура конвейера повторной идентификации обычно выстраивается похожим образом, описание и пример схемы представлены в исследовании [5]. В указанной статье система повторной идентификации рассматривается как последовательность этапов получения наблюдения, выделения объекта, формирования признакового описания и сопоставления с ранее накопленной информацией (рис. 1).

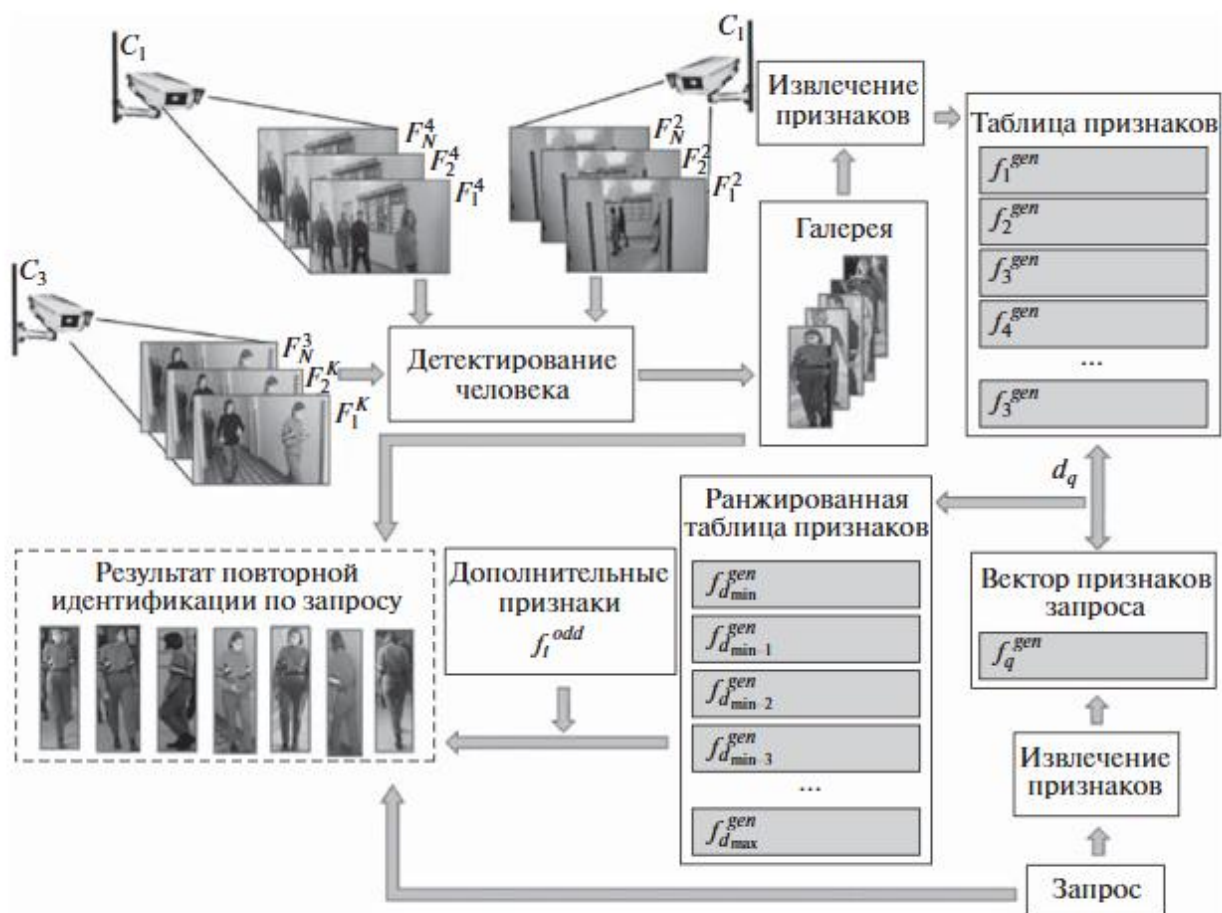


Рисунок 1 – Общая схема системы повторной идентификации [5]

В рамках данной работы эта схема адаптирована к прикладному сценарию обработки видеопотока в реальном времени. Проектируемый программный модуль включает следующие основные подсистемы:

- подсистему получения видеокадров;
- подсистему обнаружения объектов;
- подсистему локального трекинга;
- подсистему извлечения признаков;
- подсистему хранения и обновления галереи идентичностей;
- подсистему сопоставления с накопленной галереей;
- подсистему сохранения состояния и регистрации метрик.

Обобщенная логика работы системы может быть описана следующим образом: на вход поступает видеокадр, на первом этапе система обнаружения определяет ограничивающие прямоугольники объектов целевого класса, на

втором этапе подсистема локального трекинга связывает полученные обнаружения с локальными траекториями, действующими в пределах временного окна наблюдения. Далее из областей интереса формируются изображения объектов, которые подаются в модуль извлечения признаков. Для каждого объекта строится вектор признаков фиксированной размерности, после чего он сравнивается с галереей ранее накопленных представлений. Если близость к одному из существующих прототипов превышает заданный порог и подтверждается на протяжении нескольких последовательных наблюдений, объекту присваивается ранее созданный глобальный идентификатор. Если достоверное совпадение не найдено, создается новая запись в галерее.

Принципиально важным свойством спроектированной архитектуры является разделение локального и глобального уровней идентификации. Локальная идентификация существует только в рамках работы подсистемы трекинга и используется для поддержания краткосрочной целостности траектории. Глобальная идентификация, напротив, хранится в галерее и может быть повторно присвоена объекту после его исчезновения и повторного появления в кадре. Такое разделение позволяет реализовать долгосрочную повторную идентификацию без необходимости усложнять локальный трекинг, а также дает возможность независимо заменять используемый метод краткосрочного связывания объектов.

Дополнительной особенностью архитектуры является конфигурационный подход к управлению конвейером. Параметры обнаружения объектов, локального трекинга, галереи, источника видеоданных и режима сохранения результатов должны задаваться через внешний конфигурационный профиль. Это позволяет использовать одну и ту же архитектуру для серии воспроизводимых запусков с различными значениями порогов, моделями обнаружения и модулями извлечения признаков, позволяя исследовать результаты системы и подбирать параметры в соответствии со сценарием использования.

2.3.1 Система обнаружения объектов

Подсистема обнаружения предназначена для выделения объектов интереса в каждом входном кадре и формирования набора ограничивающих прямоугольников, которые далее используются на стадиях локального трекинга и извлечения признаков. В рассматриваемом программном модуле в качестве основы этой подсистемы выбрана модель семейства YOLO [14,25]. Выбор YOLO обусловлен несколькими причинами. Во-первых, данное семейство моделей сочетает высокую скорость обработки и приемлемую точность, что важно для работы в составе конвейера реального времени [25]. Во-вторых, модели этого семейства допускают подбор архитектуры и весов под разные ограничения по быстродействию и качеству. Не менее важным критерием является то, что результат обнаружения легко привести к единому внутреннему формату, удобному для последующей обработки.

На этапе проектирования предполагается, что подсистема обнаружения получает на вход отдельный видеокادر и возвращает набор локализованных объектов, для каждого из которых определяются координаты ограничивающего прямоугольника, оценка уверенности и метка класса. На этапе настройки системы должны задаваться тип применяемой модели, используемые веса, порог уверенности и перечень допустимых классов. В базовом сценарии система ориентирована на обработку только класса "человек". Благодаря единому внутреннему формату остальная архитектура не зависит от конкретной реализации этой подсистемы: при сохранении структуры выходных данных допускается последующая замена модели YOLO на другую совместимую модель локализации без переработки логики трекинга, извлечения признаков и сопоставления с галереей.

Следует отметить, что подсистема обнаружения в данной архитектуре не выполняет функции идентификации. Ее задача ограничивается локализацией объектов и подготовкой входных областей интереса для последующих этапов. Тем не менее качество обнаружения напрямую влияет на устойчивость всего конвейера, поскольку ошибки локализации приводят

либо к пропуску объекта, либо к ухудшению качества признакового описания, формируемого на этапе повторной идентификации.

2.3.2 Система локального трекинга

Подсистема трекинга в разработанном модуле используется как вспомогательный механизм локальной временной связности. Она не решает задачу долгосрочной идентификации самостоятельно, а подготавливает устойчивую последовательность наблюдений, на основе которой модуль ReID принимает решение о присвоении глобального идентификатора. В проектируемой системе предлагается использовать трекер на основе меры пересечения ограничивающих прямоугольников IoU (Intersection over Union), принцип отображен на рис. 2.

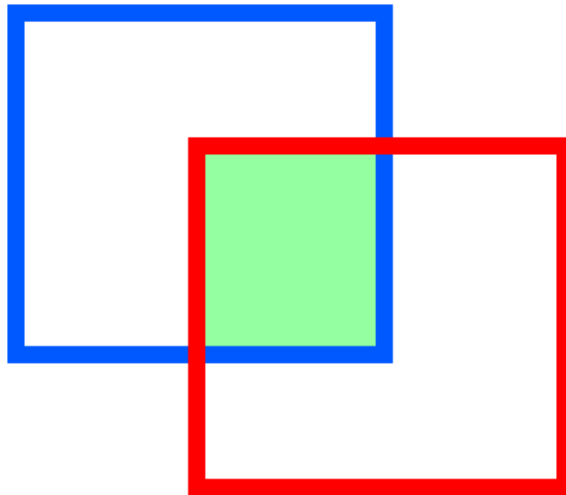


Рисунок 2 – Intersection over Union

Для двух прямоугольников B_a и B_b мера IoU вычисляется по формуле (4)

$$IoU(B_a, B_b) = \frac{|B_a \cap B_b|}{|B_a \cup B_b|}. \quad (4)$$

Если значение IoU между детекцией текущего кадра и существующим треком превышает заданный порог τ_{iou} , считается, что детекция может быть сопоставлена с данным треком. Далее все допустимые сопоставления сортируются по убыванию IoU, после чего выбираются непересекающиеся назначения по схеме жадного алгоритма.

Каждая локальная траектория в системе должна хранить локальный идентификатор, ограничивающий прямоугольник, класс объекта, оценку уверенности и счетчик пропущенных кадров. Если траектория не получает подтверждающего обнаружения в течение заранее заданного числа кадров, она удаляется. Таким образом, алгоритм трекинга отвечает только за ведение объекта в ограниченном временном интервале. Дополнительным уровнем устойчивости служит требование подтверждения решения о повторной идентификации. Если для объекта найден кандидат на глобальную идентичность, идентификатор не присваивается немедленно, а должен быть подтвержден на нескольких последовательных наблюдениях одной и той же локальной траектории. Такой подход уменьшает вероятность ложного срабатывания повторной идентификации в случае кратковременных шумов, ошибок локализации или случайного попадания похожего объекта в кадр.

Подсистема локального трекинга в выбранной архитектуре намеренно упрощена по сравнению с внешними МОТ-решениями. Это сделано для уменьшения вычислительной стоимости системы и более прозрачного управления поведением модуля идентификации. В проектируемой системе алгоритм трекинга не заменяет модуль повторной идентификации, а работает совместно с ним: локальная траектория стабилизирует последовательность наблюдений, а долгосрочное решение о том, соответствует ли объект ранее наблюдаемой идентичности, принимается только после анализа и сопоставления с галереей признаков.

2.3.3 Подсистема извлечения признаков

После выделения объекта и связывания его с локальной траекторией система переходит к формированию признакового представления. В качестве базового инструментария для подсистемы извлечения признаков предлагается использовать библиотеку Torchreid [22]. Данный выбор обусловлен тем, что библиотека ориентирована на задачи повторной идентификации, поддерживает набор готовых признаковых архитектур и предоставляет средства подключения предобученных либо пользовательских

весов. В официальном репозитории Torchreid поддерживаются как общеупотребимые сверточные архитектуры семейства ResNet, ResNeXt, DenseNet, SEnet, Inception и Xception, так и специализированные модели для повторной идентификации, включая MuDeep, HACNN, PCB, MLFN, OSNet и OSNet-AIN.

Такой подход обладает рядом преимуществ. Во-первых, использование специализированной библиотеки повторной идентификации позволяет не фиксировать архитектуру модуля извлечения признаков на этапе проектирования и оставить возможность сравнительного исследования нескольких моделей в рамках одной системы и дальнейшего определения необходимой конфигурации. Во-вторых, наличие механизма загрузки пользовательских весов делает возможным применение того же подхода не только к идентификации людей, но и к другим категориям объектов при наличии соответствующих обучающих данных и подготовленной модели признакового описания. В-третьих, пакетная обработка объектов одного кадра делает данный этап совместимым с кадровым конвейером видеоаналитики и позволяет ускорить работу сверточной нейронной сети при использовании видеокарт за счет параллельной обработки кадров полученного пакета.

В рамках настоящего исследования основное внимание сосредоточено на сценарии повторной идентификации людей, поскольку именно для этого класса объектов библиотека Torchreid предоставляет наиболее развитую экосистему моделей, предобученных весов и экспериментальных протоколов. При этом система должна поддерживать применение повторной идентификации и для других объектов за счет загрузки пользовательских весов используемой модели.

Полученные эмбединги далее нормируются по L_2 -норме. Такая нормировка переводит признаки на единичную гиперсферу и позволяет использовать скалярное произведение как косинусную меру близости между векторами признаков. Благодаря этому этап сопоставления с галереей

сводится к вычислению косинусного сходства между текущим наблюдением и набором прототипов идентичностей.

2.3.4 Галерея идентичностей и механизм сопоставления

Центральным элементом архитектуры долгосрочной идентификации является галерея идентичностей. На этапе проектирования галерея рассматривается как отдельная структура данных, содержащая глобальные идентификаторы, их прототипы в признаковом пространстве, метаданные наблюдений и служебную информацию о состоянии системы.

Для каждой идентичности в галерее хранятся:

- уникальный глобальный идентификатор;
- прототипный вектор признаков;
- класс объекта;
- время создания записи;
- время последнего наблюдения;
- время последнего обновления;
- число наблюдений и число обновлений прототипа.

Поскольку признаки предварительно нормируются по L_2 -норме, сходство между текущим эмбедингом e и прототипом p_i вычисляется по формуле (5)

$$s_i = p_i^T e, \quad (5)$$

где $s(e, p_j)$ — значение сходства; e — нормированный признаковый вектор текущего наблюдения; p_j — нормированный прототип j -й идентичности в галерее. При этом сопоставление должно выполняться только среди записей того же класса объекта, а одна и та же глобальная идентичность не должна назначаться сразу нескольким объектам одного кадра. Если максимальное значение сходства превышает порог τ_{sim} , соответствующая запись рассматривается как кандидат на повторную идентификацию. В противном случае считается, что соответствия в галерее не найдено. При этом совпадение по одному кадру не считается достаточным

основанием для немедленного присвоения глобального идентификатора. Кандидат должен быть подтвержден на протяжении нескольких последовательных наблюдений одного и того же локального трека. Только после этого объекту назначается существующий глобальный идентификатор. Если подтверждения не происходит, система продолжает рассматривать объект как потенциально новый.

Если сопоставление с галереей не дало результата и это состояние также подтверждено несколько раз подряд, для объекта создается новая запись. При этом галерея выделяет следующий свободный глобальный идентификатор и сохраняет нормированный прототип нового объекта. Такой механизм позволяет отделить случайные шумовые наблюдения от действительно новых идентичностей. При повторном наблюдении уже известного объекта его прототип в галерее может обновляться. Обновление выполняется по правилу экспоненциального скользящего среднего (6):

$$p_i^{(t+1)} = \text{norm}(\alpha p_i^{(t)} + (1 - \alpha)e_t), \quad (6)$$

где $p_j^{(t)}$ – прежний прототип идентичности; $p_j^{(t+1)}$ – обновленный прототип; e_t – текущий признаковый вектор; α – коэффициент сглаживания; norm – L_2 -норма. Обновление производится только в том случае, если сходство между наблюдением и текущим прототипом превышает дополнительный порог обновления. Такое ограничение предотвращает деградацию прототипа при сомнительных совпадениях.

Для повышения устойчивости системы в галерее должен быть реализован контроль вместимости. При переполнении удаляется наименее приоритетная запись, определяемая по времени последнего наблюдения, числу появлений и времени последнего обновления. При этом активные идентичности, связанные с текущими треками, должны защищаться от вытеснения.

Для обеспечения долгосрочной повторной идентификации галерея идентичностей должна загружаться в начале работы, периодически

сохраняться в процессе функционирования и сохраняться при завершении сеанса. Сохраняемое состояние должно включать не только сами прототипы, но и метаданные, а также сведения о совместимости с используемой моделью признакового экстрактора. Это позволяет избежать смешивания признаков, полученных разными моделями или разными весами.

Следовательно, именно галерея идентичностей преобразует описываемую систему из краткосрочного трекера в модуль долгосрочной повторной идентификации. Она обеспечивает накопление знаний о ранее наблюдаемых объектах, повторное присвоение глобальных идентификаторов и сохранение истории работы системы между отдельными запусками.

2.4 Выводы по выбору решения

В результате проектирования была сформирована архитектура программного модуля повторной идентификации объектов в видеопотоке. На основе выводов обзора аналогов были определены функциональные и интеграционные требования к системе: поддержка видеофайлов и потоковых источников, обнаружение объектов, локальное сопровождение, извлечение признаковых представлений, сопоставление с долговременной галереей, сохранение состояния и формирование результатов работы.

В качестве общей архитектуры выбран модульный конвейер, разделяющий получение кадров, детекцию, локальный трекинг, извлечение признаков, сопоставление с галереей и регистрацию результатов. Такое разделение позволяет независимо изменять модель детекции, ReID-модель, параметры локального сопровождения и стратегию обновления галереи.

Для реализации подсистемы обнаружения обосновано использование моделей семейства YOLO, для локального сопровождения – IoU-трекера, для извлечения признаков — моделей из библиотеки Torchreid. Центральным элементом разработанной архитектуры является галерея идентичностей. Она отличает проектируемое решение от обычного краткосрочного трекера: галерея хранит глобальные идентификаторы, выполняет сопоставление

текущих признаков с накопленными прототипами, обновляет их при совпадениях и поддерживает сохранение состояния между сеансами работы.

Таким образом, в данной главе определена проектная основа для последующей реализации программного модуля и исследования его свойств.

3 Реализация программного модуля

3.1 Организация программного модуля

На основе архитектурной схемы, рассмотренной в разделе 2, был реализован программный модуль повторной идентификации объектов в видеопотоке. Реализация ориентирована на потоковую обработку кадров, но для воспроизводимости экспериментального исследования использовался режим считывания видеозаписи из файла, так все конфигурации получают одинаковую последовательность кадров, что позволяет сопоставлять результаты по численным показателям.

Внутри программного модуля выделены следующие функциональные уровни:

- уровень конфигурирования, отвечающий за загрузку параметров обработки из TOML-файла средствами Python-модуля `tomllib` [26];
- уровень получения кадров, поддерживающий чтение из видеофайла, камеры или потокового источника;
- уровень обнаружения объектов, формирующий ограничивающие прямоугольники, оценки уверенности и классы объектов;
- уровень локального сопровождения, связывающий близкие по времени наблюдения в краткосрочные траектории;
- уровень извлечения признаков, преобразующий область изображения объекта в векторное описание;
- уровень галереи идентичностей, выполняющий сопоставление текущего признака с ранее накопленными прототипами;
- уровень инструментирования, сохраняющий метрики, назначения идентификаторов, конфигурацию запуска и, при необходимости, результирующее видео.

Общая логика обработки кадра представлена на рис. 3.

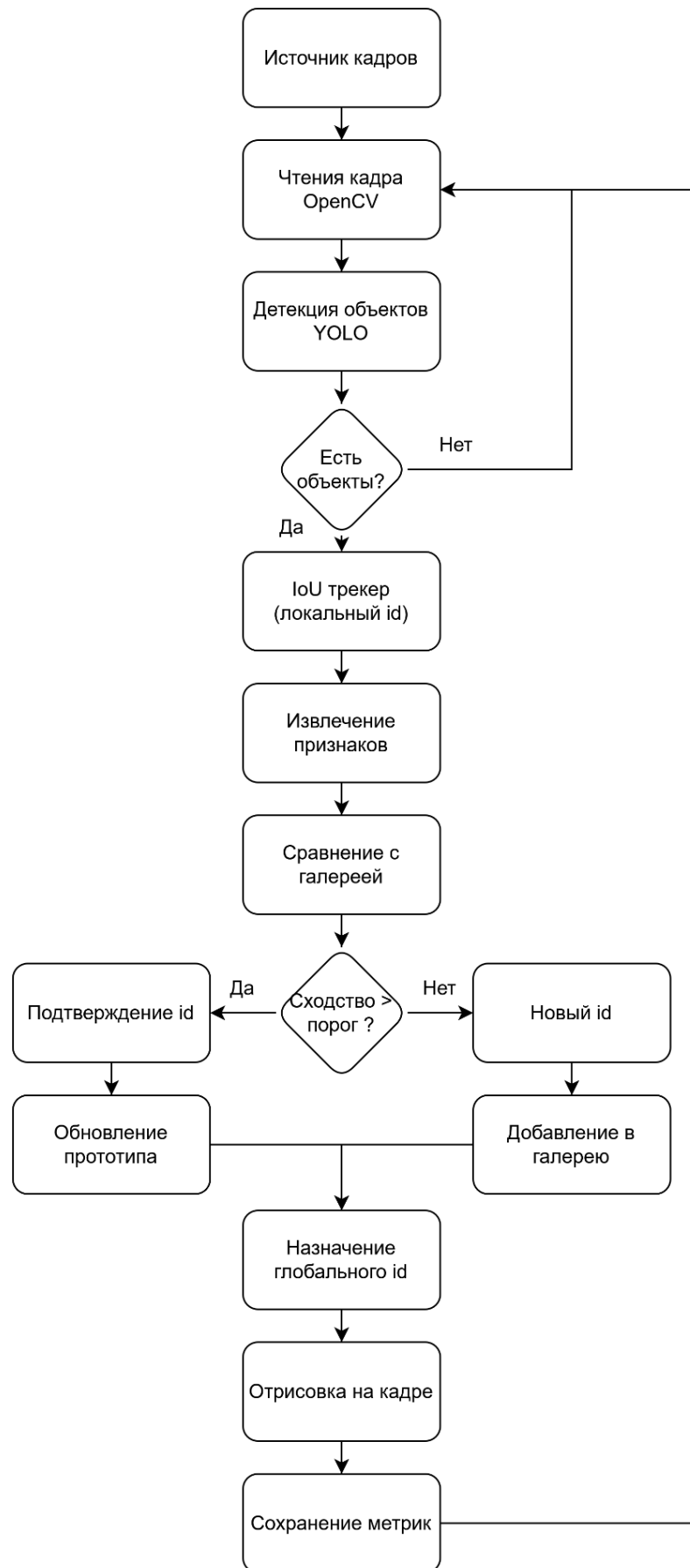


Рисунок 3 – Логика обработки кадра

Система позволяет независимо изменять модель детекции, модель извлечения признаков, пороги сопоставления, параметры локального сопровождения и обновления галереи признаков. В реализации используются механизмы сопоставления и обновления галереи, описанные в разделе 2: признаки сравниваются с прототипами той же категории объекта, решение о глобальной идентичности подтверждается на локальной траектории, а обновление прототипа выполняется только для достаточно качественных совпадений.

3.2 Архитектура программной реализации

Программный модуль реализован на языке Python, что обеспечивает удобную связку компонентов компьютерного зрения и глубокого обучения [27]. Входной и выходной видеопоток обрабатывается средствами OpenCV: библиотека используется для чтения последовательности кадров, получения параметров источника и записи результирующего видео с нанесенными идентификаторами [28,29]. В качестве входного источника поддерживаются локальные видеофайлы распространенных форматов, включая MP4 и AVI, сетевые потоки RTSP и HTTP(S)/MJPEG, а также локальные USB и встроенные камеры. Детектор объектов построен на базе Ultralytics YOLO, который выделяет в кадре области интереса выбранных классов и тем самым формирует поток наблюдений для последующей повторной идентификации [30]. Компонент извлечения признаков реализован с использованием Torchreid: предобученная ReID-модель преобразует вырезанные области объектов в признаковые векторы, по которым затем выполняется сопоставление с галереей глобальных идентификаторов. UML-диаграмма для классов обработки представлена на рис. 4.

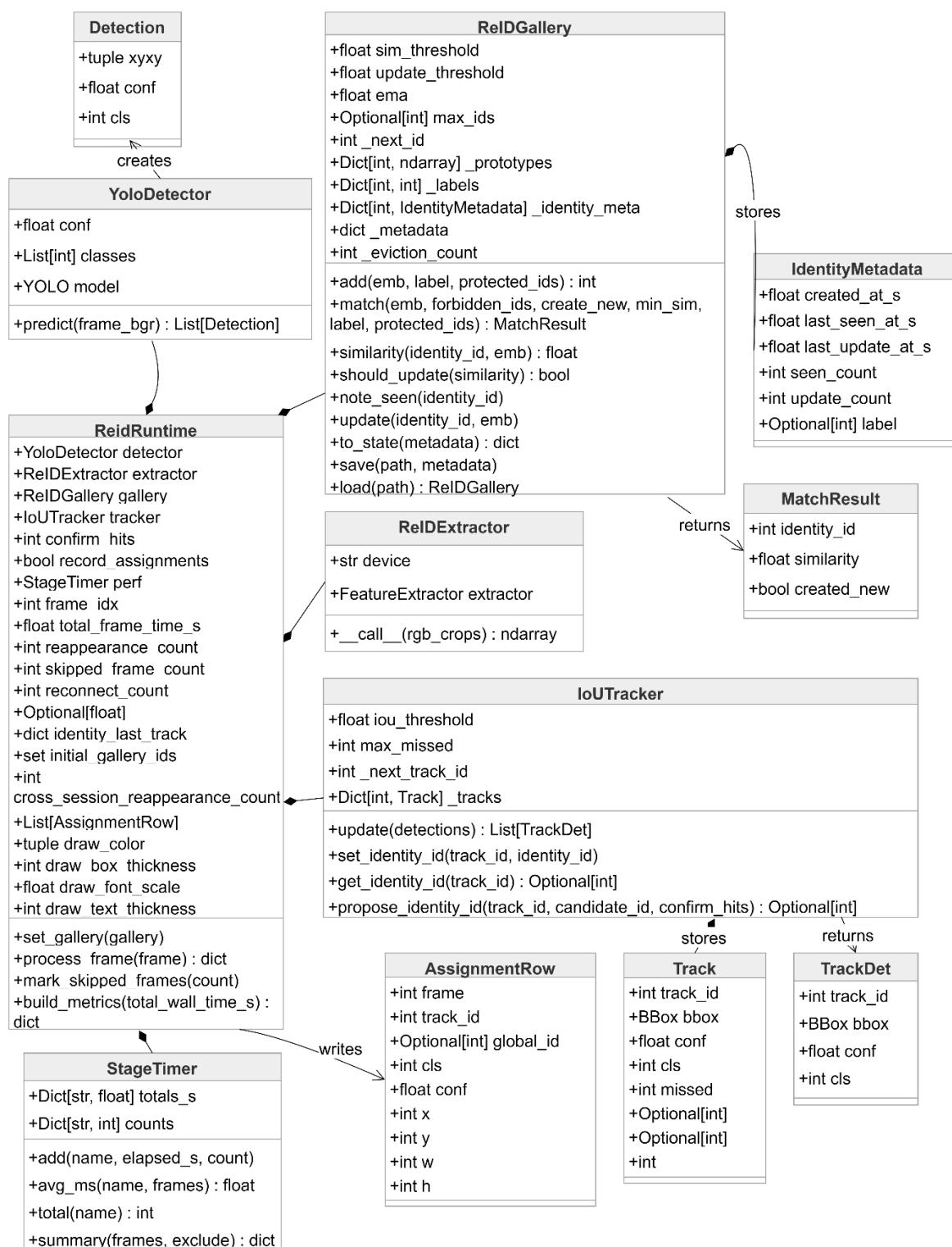


Рисунок 4 – UML-диаграмма для классов обработки кадра

Результаты работы модуля содержат агрегированные показатели производительности, структурные характеристики галереи, покадровые назначения локальных и глобальных идентификаторов, а также включают сохраненные параметры запуска. Такая реализация позволяет исследовать функциональные особенности программы, осуществлять сравнение и подбор

параметров конфигурации для заданных условий эксплуатации, включающих характеристики оборудования и условия видеозаписи.

3.3 Описание параметров конфигурации

Значение параметров конфигурации были выбраны на основе типичных значений для используемых библиотек, ограничений вычислительной платформы и предварительных запусков программного модуля. Параметры для систем видеонаблюдения, в которые будет производиться интеграция программного модуля, должны подбираться в соответствии с условиями и особенностями производимой видеозаписи, к которым можно отнести плотность сцены, количество классов анализируемых объектов, четкость кадра, наличие коллизий объектов и искажений изображения. Далее формально описаны параметры, доступные для изменения в конфигурации программы.

Главные параметры конфигурации – это модель детектора и модель извлечения признаков.

Порог уверенности детектора τ_{det} определяет минимальную нормированную оценку, при которой обнаружение допускается к дальнейшей обработке. Пусть для обнаружения d модель возвращает оценку уверенности $c_d \in [0; 1]$. Тогда обнаружение передается на следующий этап при выполнении условия (7)

$$c_d \geq \tau_{det}. \quad (7)$$

Оценка c_d является безразмерной величиной и характеризует степень согласованности найденной области с признаками выбранного класса, усвоенными при обучении. Для каждой предполагаемой ограничивающей рамки модель определяет координаты области и значения принадлежности к возможным классам. Затем из нескольких пересекающихся рамок, описывающих один и тот же объект, сохраняется рамка с наибольшей оценкой. Значение c_d , близкое к единице, дает основание считать область объектом выбранного класса; значение, близкое к нулю, указывает на слабое основание для такого решения.

Список классов детектора задает, какие категории объектов допускаются к обработке. В списке могут участвовать классы модели YOLO, соответствующие классам объектов в COCO-разметке. COCO (Common Objects in Context) представляет собой набор данных, в которой объекты размечаются по категориям и экземплярам. Этот набор широко используется для обучения и проверки моделей обнаружения объектов [31].

Для текущего признакового вектора e сначала вычисляется наибольшее значение сходства s^* среди прототипов p_i того же класса, после чего существующая глобальная идентичность рассматривается как кандидат при выполнении условия (8)

$$s^* \geq \tau_{sim}. \quad (8)$$

Порог τ_{sim} является безразмерной величиной в диапазоне $[0;1]$ и сравнивается со значением косинусной близости нормированных признаковых векторов. При интерпретации результатов меньшие значения рассматривались как более склонные к ошибочному объединению разных людей, а большие значения - как более склонные к дроблению одного человека на несколько глобальных идентификаторов.

Значение измерения τ_{upd} схоже с измерением косинусной близости, но должен иметь значение больше τ_{sim} . Параметр определяется как минимальное значение сходства s^* между текущим признаковым вектором и выбранным прототипом, при котором этот прототип разрешается обновлять в галерее признаков.

Коэффициент сглаживания α , определяющий вклад нового вектора признаков в сохраненный прототип идентификатора, был оставлен равным 0,90 для меньшего сдвига прототипа каждого идентификатора в течение проведения коротких экспериментов, так как при коротком промежутке времени объект не успевает сильно изменять свой внешний вид.

Параметры локального сопровождения задают краткосрочную связность наблюдений. Порог τ_{iou} использует меру пересечения над объединением. Для прямоугольника локальной траектории B_{trk} и

прямоугольника текущего обнаружения B_d сопоставление допускается при условии (9)

$$IoU(B_{trk}, B_d) \geq \tau_{iou}. \quad (9)$$

Для устойчивости повторной идентификации к случайным обнаружениям и излишнему вызову модели извлечения признаков были добавлены дополнительные параметры. Максимальное количество кадров, в течение которого хранится локальный трек объекта позволяет не вызывать модуль извлечения признаков, когда можно сопоставить объект геометрически, измеряя сопоставимость областей обнаружения. Количество кадров для подтверждения добавляет возможность назначать объекту идентификатор только после подтверждения его идентификации в течение указанного количества кадров. Данный параметр уменьшает количество новых идентификаторов, созданных при случайном обнаружении, например, на одном кадре, что позволяет галерее признаков не заполняться не значащими признаками идентификатора, который был ошибкой детекции и вероятнее всего больше не появится в кадре.

3.4 Сборка и сценарий использования проекта конечным пользователем

Сборка проекта осуществляется в виде стандартного Python-пакета. Исходный код размещается в каталоге `src`, а описание сборки, имя проекта, поддерживаемая версия Python, список зависимостей и точки входа фиксируются в файле `pyproject.toml`. В качестве системы сборки используется типовая схема `setuptools`, что позволяет формировать установочные артефакты без изменения алгоритмической части модуля. На выходе формируются два основных артефакта. Первый артефакт представляет собой `wheel`-пакет, предназначенный для установки готового модуля в пользовательское окружение Python. Этот формат является основным для поставки конечному пользователю, так как позволяет установить библиотеку без копирования всей исследовательской рабочей директории. Второй артефакт представляет собой архив исходного кода,

сохраняющий воспроизводимую структуру проекта и предназначенный прежде всего для разработчиков, которым требуется аудит, модификация или повторная сборка компонента.

Конечным пользователем полученной библиотеки является инженер по интеграции, разработчик прикладной видеоаналитики или технический специалист, который разворачивает и настраивает модуль в составе более крупной программной системы. Для такого пользователя важны способ установки, конфигурирования и вызова программного компонента. Поэтому вместе с библиотекой поставляются не только исполняемые артефакты, но и конфигурационный шаблон, документация по параметрам и разделение профилей зависимостей по вычислительной платформе, в частности для CPU- и GPU-вариантов установки.

После установки библиотека поддерживает два основных сценария использования. Первый сценарий является прикладным и предполагает запуск модуля как готовой утилиты командной строки. В этом режиме пользователь задает видеофайл или адрес сетевого потока, при необходимости передает внешний TOML-конфиг и получает результат в виде обработанного видео, журнала выполнения, метрик, таблицы назначений и сохраненного состояния галереи. Такой сценарий удобен при автономной проверке модуля, проведении экспериментов и подключении к существующему конвейеру видеоаналитики как внешнему исполняемому компоненту. Второй сценарий ориентирован на разработчика программной системы и предполагает использование библиотеки через импорт в собственный Python-код. В этом случае доступны функции загрузки конфигурации, запуска обработки видеофайла или живого потока, а также более низкоуровневая сборка runtime-компонентов из подсистем детекции, локального сопровождения, извлечения признаков и галереи идентичностей. Такой режим позволяет встроить разработанный модуль в серверное приложение, графический интерфейс, программный шлюз интеграции или иной контур обработки видеоданных, сохранив при этом единый интерфейс

конфигурирования и управления поведением системы. Пользователю доступно изменение конфигурации проекта после установки модуля. Модель детекции, модель ReID и другие параметры не фиксируются и доступны для изменения. Пользователь получает шаблон конфигурации, в котором может изменять тип источника видеосигнала, модель детектора, модель извлечения признаков, пути к пользовательским весам, пороги сопоставления, параметры галереи и настройки локального трекера. Благодаря такой реализации специалист по интеграции имеет возможность настроить модуль, опираясь на специфику сценариев использования.

При копировании проекта пользователю доступны модульные тесты. В текущей реализации набор тестов включает проверки загрузки встроенной конфигурации по умолчанию, частичного переопределения параметров через внешний TOML-файл, записи шаблона конфигурации в файл, определения базового каталога относительно расположения конфигурационного файла, разрешения относительных путей, автоматического определения типа источника для строк вида `rtsp://`, `http://`, `https://`, файлового пути и индекса камеры, корректного объединения нескольких наборов конфигурационных переопределений, а также генерации исключения при передаче недопустимого значения порога детектора. Указанные проверки выполняются без загрузки нейросетевых весов, без открытия видеисточника и без использования GPU, поэтому они позволяют сразу после установки библиотеки проверить работоспособность конфигурационной подсистемы, согласованность публичного интерфейса и корректность базовой логики запуска, не переходя к полному экспериментальному запуску на видеоданных.

3.5 Выводы по результатам разработки

Разработанный модуль реализует полный конвейер повторной идентификации: от получения кадра и обнаружения объектов до сопоставления с галереей, сохранения результатов и повторного использования идентификаторов между запусками.

К сильным сторонам можно отнести управляемость конфигурации, воспроизводимость запусков, сохранение детальных метрик и возможность анализа поведения галереи. Выбранная конфигурация восстанавливает общий объем наблюдений относительно разметки и достигает взвешенной чистоты глобальных идентификаторов. Она также демонстрирует работоспособность механизма долгосрочного сопоставления: часть повторных появлений после разрыва получает прежний глобальный идентификатор.

Дальнейшее развитие модуля целесообразно вести по нескольким направлениям. На архитектурном уровне перспективны хранение нескольких признаков состояний одной идентичности для разных ракурсов и условий наблюдения, замена текущего локального сопровождения на более устойчивый трекер, совместно учитывающий геометрию и внешний вид объекта, разделение порогов внутрисеансового и межсеансового сопоставления, а также расширение автоматической оценки по метрикам IDF1 и NOTA, что должно повысить стабильность и точность работы системы [9,13,32,33]. Следующим шагом является добавление обработки нескольких источников видеосигнала и переход к многокамерной повторной идентификации, где помимо признакового описания требуется учитывать межкамерные различия и пространственно-временные связи между наблюдениями [7,19,21]. Для повышения производительности при практическом внедрении целесообразен перевод используемых моделей в формат ONNX и выполнение вывода через ONNX Runtime, поскольку ONNX предоставляет переносимое представление модели, а ONNX Runtime поддерживает графовые оптимизации и аппаратно-зависимое ускорение [34,35]. Также немаловажным улучшением для долговременного хранения галереи признаков, журналирования ее изменений и создания резервных копий является возможность подключения базы данных; например, использование SQLite позволяет выполнять согласованное онлайн-резервное копирование без длительной блокировки рабочей базы [36].

4 Исследование свойств решения

4.1 Экспериментальные данные и вычислительные условия

Для исследования работоспособности построенного программного модуля был проведен ряд экспериментов. Для тестирования системы в качестве экспериментальных данных были выбраны видеофрагменты из набора данных MOT17. Бенчмарк MOTChallenge предназначен для стандартизированной оценки алгоритмов многообъектного сопровождения в однокамерных видеопоследовательностях [37]. В MOT17 используются последовательности с уточненной разметкой, содержащей информацию об обнаруженных объектах для каждого кадра. В настоящей работе публичные детекции MOT17 не использовались как вход алгоритма: из набора были взяты видеокадры и эталонная разметка, при этом обнаружение объектов в видеопоследовательности выполнялось с помощью моделей YOLO, используемых в модуле детекции построенного конвейера обработки.

Конфигурационные параметры подбирались на основе входного видеофрагмента MOT17-11-SDP. Последовательность содержит сцену с большим количеством людей, взаимными перекрытиями, изменением масштаба объектов и плотным распределением фигур в кадре. В разметке MOT17 для данной последовательности присутствуют все аннотации, однако при оценке использовался фильтр: учитывались только записи класса "person" с видимостью не ниже 0,25. После такой фильтрации для 600 кадров было получено 9393 эталонных наблюдения и 53 уникальные идентичности.

Экспериментальная проверка проводилась на ноутбуке с графическим ускорителем. Параметры вычислительной системы приведены в таблице 2.

Таблица 2 – Параметры оборудования

Параметр	Значение
Процессор	AMD Ryzen 5 5600H with Radeon Graphics
Количество физических ядер	6

Продолжение таблицы 2

Параметр	Значение
Количество логических потоков	12
Оперативная память	16 ГБ
Графический ускоритель	NVIDIA GeForce RTX 3050 Laptop GPU
Объем видеопамяти	4 ГБ
Версия PyTorch	2.11.0+cu128
Версия CUDA	12.8

Параметры используемых видеофрагментов приведены в таблице 3.

Таблица 3 – Параметры входных данных

Данные	Количество кадров	Частота кадров	Разрешение	Назначение
MOT17-11-SDP	900	30 кадр/с	1920x1080	Подбор параметров конфигурации
MOT17-05-SDP	200	14 кадр/с	640x480	Моделирование пропажи объекта из кадра и последующего возвращения
Тестовый видеофрагмент без разметки	3755	30 кадр/с	1280x720	Одновременная детекция людей и автомобилей

4.2 Проверка функциональных возможностей

Для определения соответствия разрабатываемого решения требованиям к функциональности проведены эксперименты, проверяющие сохранение галереи между запусками, работу с несколькими классами объектов и возможность подключения пользовательских весов модели признакового описания.

4.2.1 Повторная идентификация после временной потери объекта

Для проверки сохранения и повторного использования галереи смоделирован сценарий, в котором человек временно отсутствует в обрабатываемой последовательности. Эксперимент проводился на двух фрагментах MOT17-05-SDP. В первом фрагменте система обработала 100 кадров и сохранила сформированную галерею. Затем из исходной последовательности пропущен промежуток в 30 кадров, что при частоте 14 кадр/с соответствует примерно 2,14 секунды. Второй фрагмент также содержал 100 кадров, но обрабатывался уже с загрузкой ранее сохраненной галереи.

Кадр из первого фрагмента показан на рис. 5. На этом этапе система формирует начальные записи галереи и связывает текущие локальные траектории с глобальными идентификаторами.

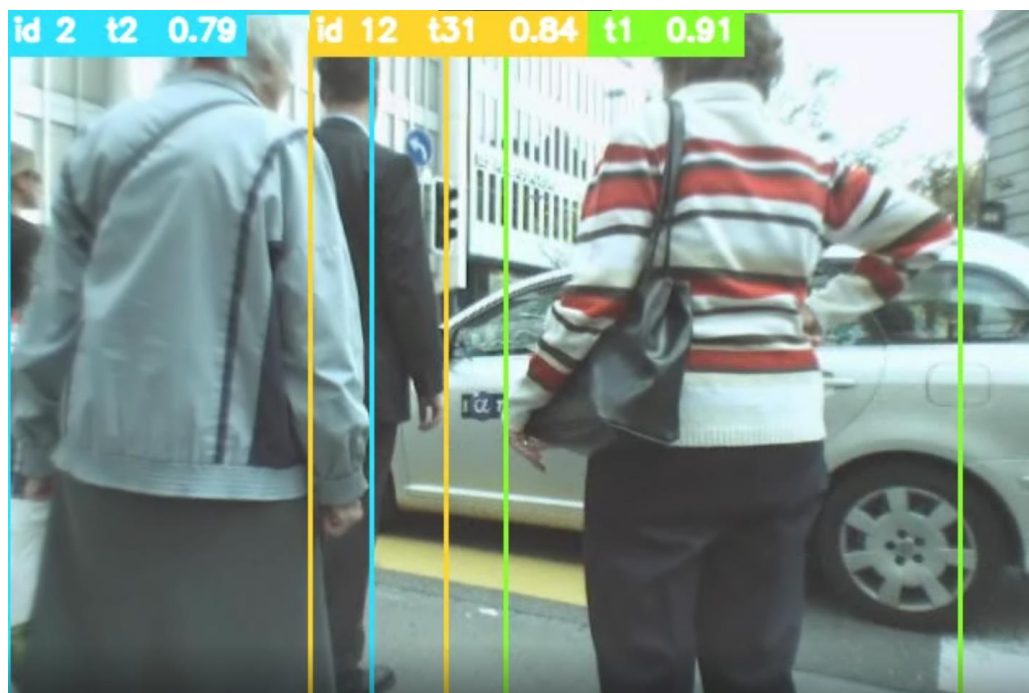


Рисунок 5 – Формирование галереи идентификаторов до пропуска фрагмента

Результат второго фрагмента показан на рис. 6. После загрузки сохраненной галереи часть объектов получает идентификаторы, созданные в предыдущем запуске. Это подтверждает, что галерея не является только временной структурой внутри одного процесса, а может использоваться как состояние для долгосрочной идентификации.

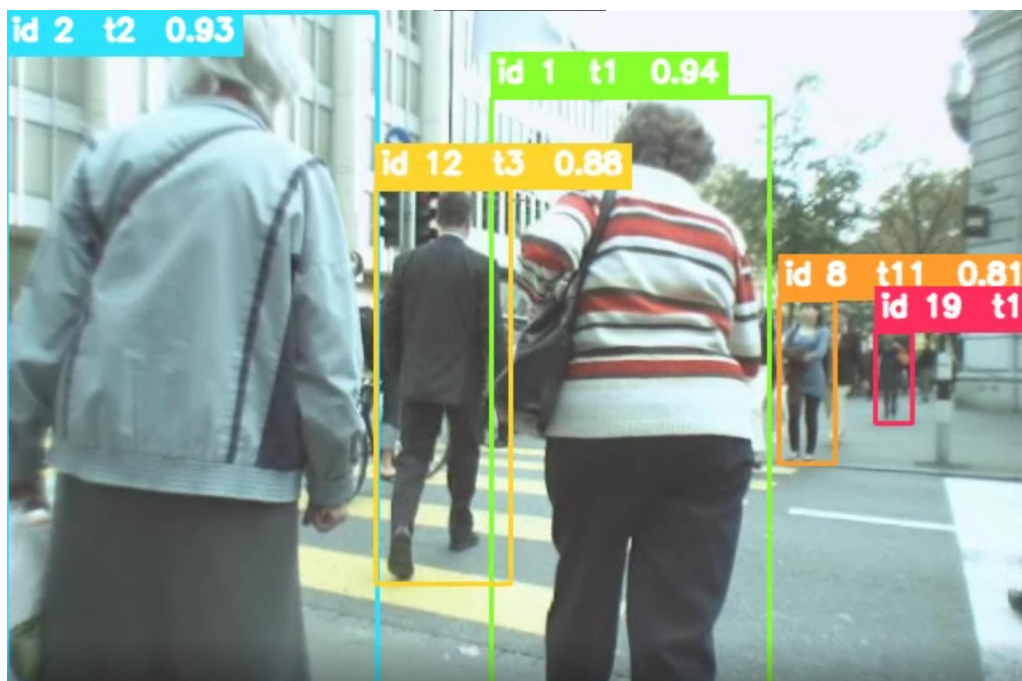


Рисунок 6 – Формирование галереи идентификаторов до пропуска фрагмента

В первом фрагменте обработано 100 кадров, сформировано 805 наблюдений и создано 15 глобальных идентификаторов. Средняя скорость составила 14,85 кадр/с при частоте источника 14 кадр/с. Во втором фрагменте обработано 100 кадров, сформировано 908 наблюдений и создано 24 глобальных идентификатора с учетом загруженной галереи. При этом зафиксировано 9 межсеансовых повторных связываний, что подтверждает возможность повторного использования накопленной галереи после разрыва обработки. Программный модуль выполняет требования к долгосрочной идентификации, сохраняя согласованность глобальных идентификаторов между независимыми запусками. На рисунках видно, что объектам, появившимся во втором фрагменте, сопоставляются те же идентификаторы, которые присвоены им в предыдущем запуске. При этом скорость обработки кадров для данных условий видеозаписи позволяет использовать потоковую обработку.

4.2.2 Работа с несколькими классами объектов

Следующий эксперимент проверял возможность задавать несколько классов объектов на входе детектора. Для модели YOLO выбраны классы 0 и 2, соответствующие человеку и автомобилю. Остальная часть конвейера не

изменялась: для найденных областей формировались признаки, выполнялось сопоставление с галереей и отображались глобальные идентификаторы.

Примеры результата приведены на рис. 7 и рис. 8.



Рисунок 7 – Детекция и идентификация людей и автомобилей, первый пример



Рисунок 8 – Детекция и идентификация людей и автомобилей, второй пример

Видеоролик содержал 3755 кадров с частотой 30 кадр/с, в ходе запуска обработано 3752 кадра. Система сформировала 9491 наблюдение, среднее количество наблюдений составило 2,53 объекта на кадр. Средняя скорость обработки достигла 21,42 кадр/с, а отношение скорости обработки к частоте источника составило 0,71.

Эксперимент показывает, что модуль обнаружения не зафиксирован на одном классе и позволяет анализировать и идентифицировать любые доступные для детектора объекты. В то же время используемая базовая модель OSNet ориентирована прежде всего на признаки человека, поэтому полноценная ReID-оценка для транспортных средств требует специализированных весов или отдельной модели. Это согласуется с документацией Torchreid, в которой доступные предобученные модели представлены как модели для задачи повторной идентификации людей [30]. Важно отметить, что несмотря на общий список идентификаторов, сравнение векторов признаков разных классов производится только с объектами того же класса, то есть автомобилю не может присвоиться идентификатор, присвоенный ранее человеку.

4.2.3 Загрузка пользовательских весов модели

Отдельно проверялась возможность использовать пользовательские веса модели извлечения признаков. Для этого выполнялся запуск с весами модели, предварительно обученной в течение 100 эпох на наборе Market-1501 [38]. Market-1501 является одним из широко используемых наборов данных для повторной идентификации людей и содержит изображения одних и тех же идентичностей, полученные разными камерами, что делает его типовым источником для проверки моделей повторной идентификации человека.

Данный эксперимент направлен на проверку программной интеграции. Результат показал, что внешний файл весов может быть загружен и использован без изменения логики детекции, локального сопровождения и галереи. При наличии предметно-ориентированного датасета, направленного на идентификацию не только людей, но и других объектов, ReID-компонент

можно дообучить, сохранив общий интерфейс конвейера и обеспечив возможность всестороннего анализа.

4.3 Метрики экспериментальной оценки

Оценка результатов экспериментов строится на двух группах показателей. Первая группа относится к стандартной логике оценки многообъектного сопровождения: сопоставление с эталонными прямоугольниками, полнота, точность, переключения идентификаторов и качество локализации. Подобные группы ошибок лежат в основе метрик трекинга [39], ID-метрик, включая IDF1 [32], а также HOTA, где отдельно учитываются ошибки детекции, ассоциации и локализации [33]. Вторая группа относится к поведению галереи: число глобальных идентификаторов, повторные связывания, межсеансовое восстановление и устойчивость структуры галереи.

Для сопоставления результата системы с разметкой каждый кадр обрабатывался независимо. Пусть G_f обозначает множество размеченных прямоугольников на кадре f , а A_f обозначает множество прямоугольников, сформированных системой. Для каждого кадра вводилось множество допустимых соответствий (10)

$$E_f = \{(g, a) \in G_f \times A_f \mid IoU(g, a) \geq 0,5\}, \quad (10)$$

где $IoU(g, a)$ обозначает коэффициент пересечения по объединению для прямоугольников g и a . Из множества E_f выбиралось подмножество $M_f \subseteq E_f$, удовлетворяющее условию взаимной однозначности: каждый элемент $g \in G_f$ и каждый элемент $a \in A_f$ могли входить не более чем в одно соответствие. Если на кадре существовало несколько допустимых вариантов, предпочтение отдавалось соответствиям с большими значениями IoU . В результате для каждого кадра строилось непересекающееся множество соответствий, максимизирующее качество геометрического согласования между разметкой и результатом системы. Такой способ сопоставления

соответствует принятой практике оценки качества многообъектного сопровождения [37,39].

Для оценки детекции использовались следующие величины (11):

$$N_{GT} = \sum_f |G_f|, \quad N_{SYS} = \sum_f |A_f|, \quad N_{TP} = \sum_f |M_f|. \quad (11)$$

На их основе вычислялись полнота и точность (12):

$$Recall = \frac{N_{TP}}{N_{GT}}, \quad Precision = \frac{N_{TP}}{N_{SYS}}, \quad (12)$$

где TP – число корректно сопоставленных обнаружений; FP – число обнаружений, не подтвержденных разметкой; FN – число объектов разметки, для которых система не сформировала корректное обнаружение. Здесь $Recall$ показывает долю размеченных объектов, для которых система сформировала достаточно точную локализацию, а $Precision$ показывает долю прямоугольников системы, подтвержденных разметкой. Для совместного измерения $Recall$ и $Precision$ использовалась метрика F_1 (13):

$$F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}. \quad (13)$$

Этот показатель учитывает совокупное влияние $Recall$ и $Precision$ и позволяет проводить сравнение по единому значению.

Для анализа идентичностей рассматривались пары (g, y) , где g обозначает эталонную идентичность, а y обозначает глобальный идентификатор, назначенный системой. Пусть $n_{y,g}$ обозначает число совпавших наблюдений, в которых системный идентификатор y был сопоставлен с эталонной идентичностью g . Тогда чистота системного идентификатора вычислялась как (14)

$$Purity(y) = \frac{\max_g n_{y,g}}{\sum_g n_{y,g}}. \quad (14)$$

Метрика чистоты заимствует идею внешней оценки кластеризации, где каждый кластер сравнивается с известными классами, а его качество определяется долей наиболее представленного эталонного класса [40]. В

данной работе системный глобальный идентификатор рассматривается как кластер наблюдений, а эталонная идентичность MOT17 - как известная принадлежность наблюдения.

Средняя чистота вычислялась как обычное среднее по всем системным идентификаторам, сопоставленным с разметкой (15):

$$Purity_{mean} = \frac{1}{|Y|} \sum_{y \in Y} Purity(y). \quad (15)$$

Взвешенная чистота учитывала вклад идентификаторов пропорционально числу наблюдений (16):

$$Purity_{weighted} = \frac{\sum_{y \in Y} \max_g n_{y,g}}{\sum_{y \in Y} \sum_g n_{y,g}}. \quad (16)$$

Различие между этими метриками принципиально. Средняя чистота одинаково учитывает короткие и длинные идентификаторы, поэтому может быть высокой даже тогда, когда несколько крупных идентификаторов объединяют разных людей. Взвешенная чистота сильнее отражает качество на массовых наблюдениях и поэтому была использована как основной показатель при выборе конфигурации.

Кроме чистоты, фиксировались следующие структурные показатели:

- число глобальных идентификаторов системы N_{ID}^{SYS} и число эталонных идентичностей N_{ID}^{GT} ;
- число фрагментированных эталонных идентичностей, то есть случаев, когда один человек получил более одного глобального идентификатора системы;
- число объединенных системных идентификаторов, то есть случаев, когда один глобальный идентификатор системы соответствовал нескольким эталонным идентичностям;
- среднее число глобальных идентификаторов на одну эталонную идентичность;

- среднее число эталонных идентичностей на один глобальный идентификатор системы;
- число переключений идентификатора $IDSW$, когда для одной эталонной идентичности в последовательности совпавших наблюдений меняется глобальный идентификатор системы;
- доля успешных повторных появлений после разрыва, превышающего допустимую длину сохранения локального трека.

Показатель повторного появления был введен как прикладная метрика долгосрочной повторной идентификации. Если два последовательных совпавших наблюдения одной эталонной идентичности разделены промежутком, превышающим заданное количество кадров, в течение которого может сохраняться локальный трек, событие считается повторным появлением. Оно считается успешным, если после такого разрыва система назначила тот же глобальный идентификатор, что и до разрыва (17):

$$R_{reapp} = \frac{N_{same}}{N_{same} + N_{changed}}. \quad (17)$$

Данная метрика показывает долю успешных повторных наблюдений, N_{same} – количество случаев, когда назначен тот же id, $N_{changed}$ – когда id сменился после повторного появления объекта.

Для анализа производительности фиксировались средняя скорость обработки, среднее время обработки кадра и средние задержки отдельных этапов. Отношение скорости обработки к частоте источника вычислялось как (18)

$$R_{src} = \frac{FPS_{proc}}{FPS_{source}}. \quad (18)$$

Если $R_{src} \geq 1$, обработка успевает за исходным видеопотоком. Если $R_{src} < 1$, система в текущей конфигурации работает медленнее реального времени для данного источника.

4.4 Экспериментальная оценка устойчивости повторного назначения идентификатора

Для количественной оценки свойств разработанного программного модуля был использован видеофрагмент MOT17-11-SDP длиной 900 кадров. Данная последовательность характеризуется высокой плотностью пешеходного потока, частыми окклюзиями и длительными взаимными перекрытиями объектов, поэтому позволяет оценить поведение системы в неблагоприятных условиях наблюдения

Для проведения экспериментов были зафиксированы модели детекции и извлечения признаков. В качестве модели детекции выбрана модель yolo26n, как передовая модель семейства YOLO. В качестве нейросетевой модели извлечения признаков была зафиксирована OSNet x1.0 из библиотеки Torchreid [15,22,41]. Такой выбор связан с тем, что OSNet специально разработана для повторной идентификации людей и в сводных таблицах Torchreid показывает более выгодное соотношение точности и вычислительной стоимости, чем универсальные сверточные архитектуры. В режиме внутридоменной повторной идентификации ReID модель OSNet x1.0 при 2,2 млн параметров и 0,98 GFLOPs достигает 94,2 Rank-1 и 82,6 mAP на Market1501, 87,0 Rank-1 и 70,2 mAP на DukeMTMC-reID, 74,9 Rank-1 и 43,8 mAP на MSMT17. Для сравнения, ResNet50, основанная на архитектуре остаточных связей [42], требует 23,5 млн параметров и 2,7 GFLOPs, но дает более низкие значения на тех же наборах: 87,9/70,4, 78,3/58,9 и 63,2/33,9 соответственно. Мобильные варианты MobileNetV2 [43] имеют меньшую вычислительную стоимость, однако заметно уступают OSNet по Rank-1 и mAP. В задачах междоменной идентификации, где модель проверяется на данных из домена, отличного от обучающего [44], OSNet x1.0 также превосходит ResNet50, что важно для прикладного сценария без обучения под конкретную камеру. В зависимости от оборудования и условий видеозаписи могут быть выбраны другие модели, доступные в Torchreid, а

также могут быть загружены веса предобученной средствами библиотеки модели извлечения признаков.

На первом этапе исследовалось влияние порога уверенности детектора τ_{det} . На данном этапе значения *Recall*, *Precision* и F_1 зависели только от этого параметра, поскольку остальные компоненты конвейера оставались неизменными. Результаты приведены в таблице 5.

Таблица 5 – Метрики детектора

τ_{det}	<i>Recall</i>	<i>Precision</i>	F_1
0,10	0,753	0,668	0,708
0,15	0,745	0,747	0,746
0,20	0,735	0,795	0,764
0,25	0,728	0,827	0,774
0,30	0,723	0,851	0,782
0,35	0,716	0,869	0,785
0,40	0,706	0,885	0,786
0,45	0,698	0,901	0,787
0,50	0,686	0,915	0,784
0,55	0,676	0,925	0,781
0,60	0,664	0,935	0,777
0,65	0,649	0,943	0,769

Полученные результаты показывают ожидаемый компромисс между полнотой и точностью обнаружения (рис. 9). При увеличении τ_{det} значение *Recall* монотонно снижается, тогда как *Precision*, напротив, возрастает. Максимум полноты достигается при $\tau_{det} = 0,10$, а максимум точности - при $\tau_{det} = 0,65$. В качестве опорной сводной метрики использовалось значение F_1 . Максимум $F_1 = 0,787$ был получен при $\tau_{det} = 0,45$, поэтому именно это значение было зафиксировано для дальнейшего анализа параметров повторной идентификации.

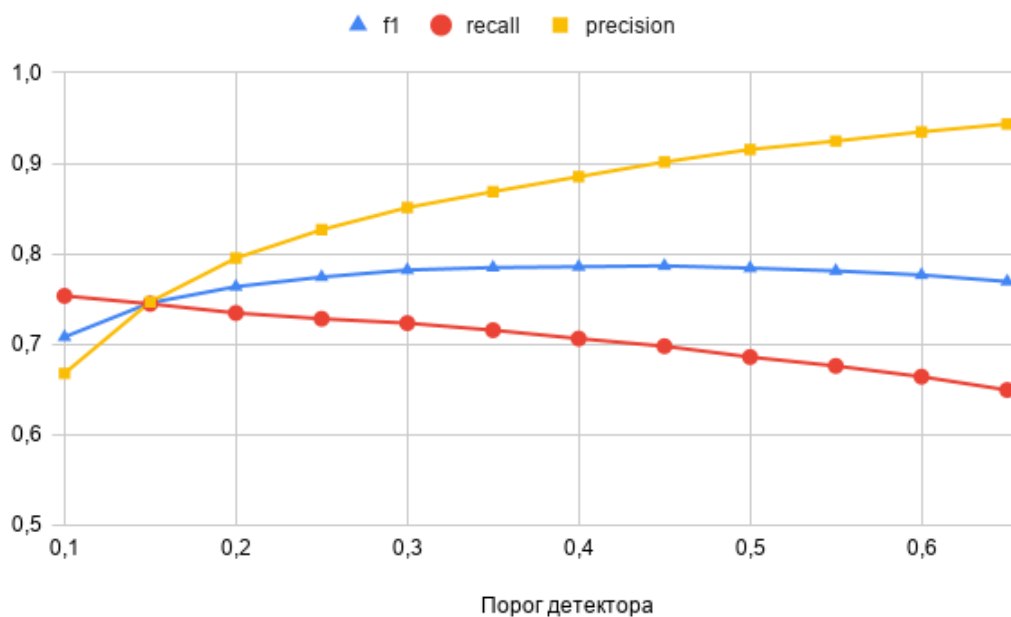


Рисунок 9 – График влияния порога детекции на метрики точности и полноты обнаружений

После фиксации порога детектора исследовалось влияние основных параметров ReID-модуля. Сначала варьировались порог сопоставления с галереей τ_{sim} и порог обновления прототипа τ_{upd} . В этой серии экспериментов ключевым критерием выступала не только чистота идентичностей, но и метрика R_{reapp} , поскольку именно она напрямую отражает способность системы вернуть объекту прежний глобальный идентификатор после разрыва наблюдения (рис. 10).

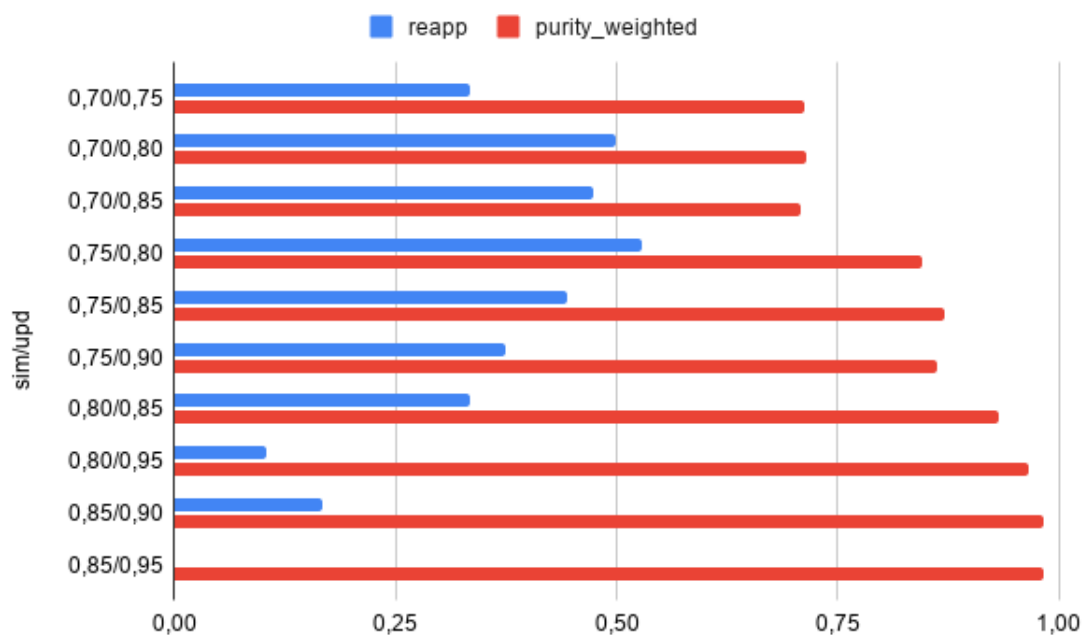


Рисунок 10 – Диаграмма значений R_{reapp} и $Purity_{weighted}$ для порогов сопоставления и обновления

Результаты показали, что ужесточение порогов сопоставления действительно повышает чистоту глобальных идентификаторов, однако этот выигрыш сопровождается ухудшением повторного назначения идентификатора. Наиболее строгая из измеренных конфигураций $\tau_{sim} = 0,85$, $\tau_{upd} = 0,95$ обеспечила $Purity_{weighted} = 0,981$, но при этом R_{reapp} снизилась до 0, то есть после разрыва наблюдения система фактически перестала возвращать объекту ранее назначенный идентификатор. Напротив, конфигурация $\tau_{sim} = 0,75$, $\tau_{upd} = 0,80$ обеспечила значение $R_{reapp} = 0,529$ при $Purity_{weighted} = 0,845$. Соответственно, эта пара порогов была выбрана как основная для дальнейшего исследования.

На следующем этапе исследовалось влияние коэффициента экспоненциального сглаживания α , определяющего скорость обновления прототипа идентичности в галерее (рис. 11).

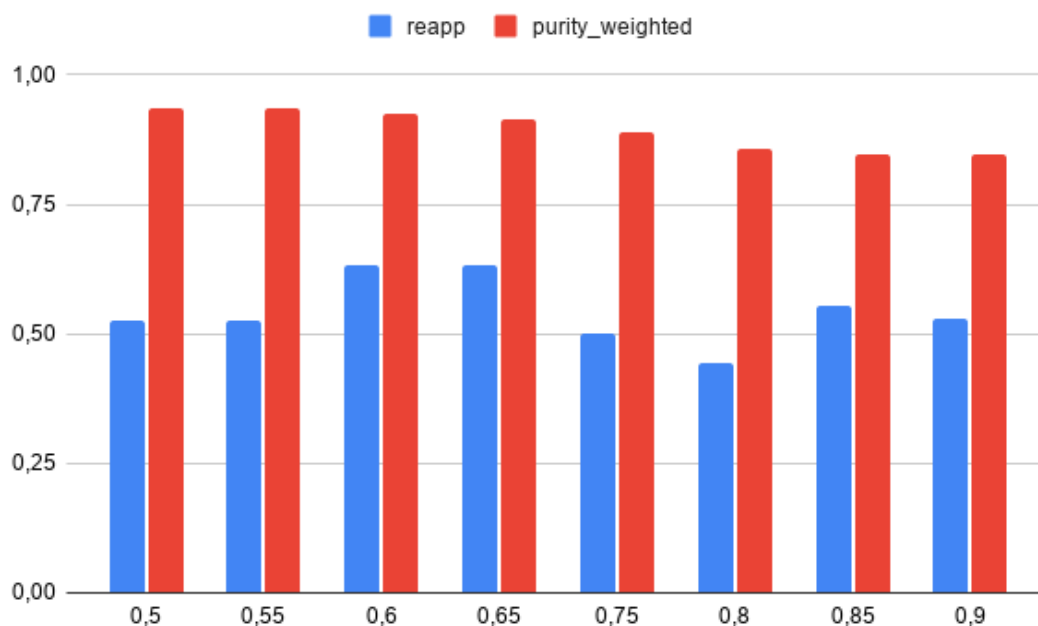


Рисунок 11 – Диаграмма значений R_{reapp} и $Purity_{weighted}$ для коэффициента экспоненциального сглаживания α

Было установлено, что максимальное значение $R_{reapp} = 0,632$ достигается при $\alpha = 0,60$ и $\alpha = 0,65$. Для дальнейшей настройки было выбрано значение $\alpha = 0,65$, поскольку при том же значении R_{reapp} оно сопровождается меньшим числом переключений идентификатора, меньшим числом ошибочных объединений и меньшей фрагментацией эталонных идентичностей. Таким образом, уменьшение α относительно исходных значений улучшило устойчивость повторного назначения идентификатора, но дальнейшее снижение ниже 0,60 уже не давало дополнительного выигрыша по ключевой метрике, при этом слишком низкое значение будет давать быструю деградацию признаков идентичности.

Завершающим основным этапом было исследование порога локального трекинга τ_{iou} (рис. 12).

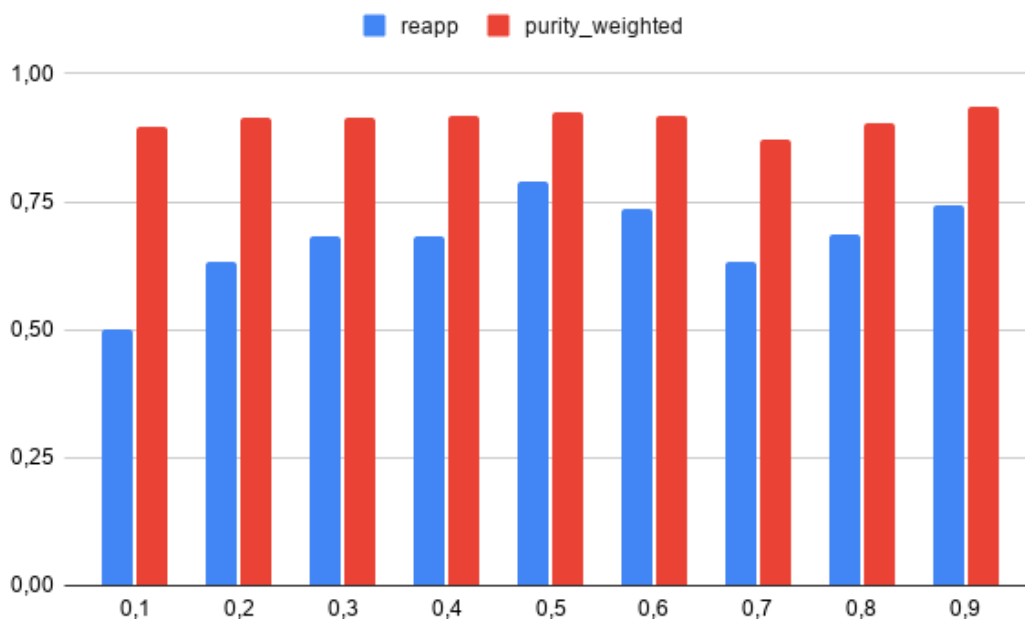


Рисунок 12 – Диаграмма значений R_{reapp} и $Purity_{weighted}$ для порога локального трекинга

Эксперименты показали, что при слишком мягком сопоставлении локальных треков ($\tau_{iou} = 0,10 - 0,20$) система чаще продолжает локальный трек по слабому геометрическому совпадению, что ухудшает качество глобального переназначения. При повышении порога до $\tau_{iou} = 0,50$ были получены значения основных метрик: $R_{reapp} = 0,790$, $Purity_{weighted} = 0,925$ и $IDSW = 11$. Дальнейшее увеличение порога до 0,80-0,90 приводило к резкому росту числа локальных треков и доли наблюдений без подтвержденного глобального идентификатора, поэтому такие режимы нельзя считать практически приемлемыми, даже если по отдельным показателям они выглядят сильными.

Итоговые параметры выбранной конфигурации приведены в таблице 6.

Таблица 6 – Выбранная конфигурация для сцены MOT17-11-SDP

Группа параметров	Значение
Модель детекции	YOLOv26
Порог уверенности детектора	0,45
Классы объектов	0
Модель извлечения признаков	OSNet x1.0

Продолжение таблицы 6

Группа параметров	Значение
Порог сопоставления	0,75
Порог обновления прототипа	0,80
Коэффициент сглаживания прототипа	0,65
Порог IoU для локального сопровождения	0,50

4.5 Результаты экспериментальной оценки разработанного программного модуля

В таблице 7 ниже приведены значения метрик, полученные при фиксированной конфигурации модуля.

Таблица 7 – Исследование метрик полученного решения

Группа показателей	Обозначение	Значение
Полнота и точность наблюдений	$Recall$	0,698
	$Precision$	0,901
	$\overline{IoU}$	0,856
Устойчивость глобальной идентификации	$Purity_{weighted}$	0,925
	$Purity_{mean}$	0,960
	N_{ID}^{SYS}	44
	N_{frag}	9
	N_{merge}	7
	$IDSW$	11
Повторная идентификация после разрыва наблюдения	R_{reapp}	0,790
	$N_{same} + N_{changed}$	19
	N_{same}	15
Производительность	FPS_{proc}	14,481
	R_{src}	0,483
	$\overline{t_{det}}$	21,624
	$\overline{t_{reid}}$	35,428
	$\overline{t_{track}}$	0,077

Как видно из таблицы 7, выбранная конфигурация обеспечивает достаточно высокое качество наблюдений и устойчивую работу механизма глобальной идентификации даже в плотной многолюдной сцене. Наиболее значимым для настоящего исследования является то, что при сохранении приемлемых значений детекторных и структурных метрик система демонстрирует высокое значение R_{reapp} .

Для выбранной конфигурации $\tau_{det} = 0,45$, $\tau_{sim} = 0,75$, $\alpha = 0,65$, $\tau_{upd} = 0,80$, $\tau_{iou} = 0,50$ были получены следующие значения: $Recall = 0,698$, $Precision = 0,901$, $F_1 = 0,787$, среднее значение IoU совпавших наблюдений составило 0,856. Эти значения показывают, что подсистема детекции формирует достаточно точный поток наблюдений для последующей долгосрочной идентификации даже в плотной сцене с частыми перекрытиями.

Более показательными для настоящего исследования являются метрики глобальной идентификации. В выбранной конфигурации система обеспечила $Purity_{mean} = 0,960$ и $Purity_{weighted} = 0,925$, что указывает на высокую содержательную однородность сформированных глобальных идентификаторов. При этом число переключений идентификатора составило $IDSW = 11$, число фрагментированных эталонных идентичностей – 9, а число ошибочных объединений – 7. В совокупности это означает, что система не только поддерживает высокую чистоту идентичностей, но и удерживает приемлемую структурную устойчивость глобального пространства идентификаторов.

Ключевой результат получен по метрике R_{reapp} . Для событий повторного появления после разрыва длительностью более 15 кадров система сохранила тот же глобальный идентификатор в 15 случаях из 19, что соответствует $R_{reapp} = 0,790$. Именно эта величина является центральным итогом проведенного исследования, поскольку она напрямую характеризует устойчивость повторного назначения глобального идентификатора объекту

после временной потери наблюдения. Важно подчеркнуть, что выбранная конфигурация обеспечивает не только высокое значение R_{reapp} , но и делает это без критического ухудшения чистоты идентичностей. Тем самым подтверждается, что искомая устойчивость достигается не за счет искусственного смягчения правил сопоставления, а за счет согласованного выбора параметров детектора, галереи признаков и локального сопровождения.

С точки зрения производительности выбранная конфигурация обеспечивает среднюю скорость обработки 14,48 кадр/с, что соответствует 0,483 от частоты исходного потока 30 кадр/с. Среднее время детекции составляет 21,62 мс на кадр, среднее время этапа повторной идентификации - 35,43 мс, а затраты на локальный трекинг - 0,08 мс. Следовательно, основная вычислительная нагрузка сосредоточена на нейросетевых компонентах конвейера, тогда как логика сопровождения и переназначения идентификаторов требует пренебрежимо малых затрат по сравнению с детекцией и извлечением признаков. Доли затрачиваемого времени на обработку кадра можно увидеть на рис. 13.

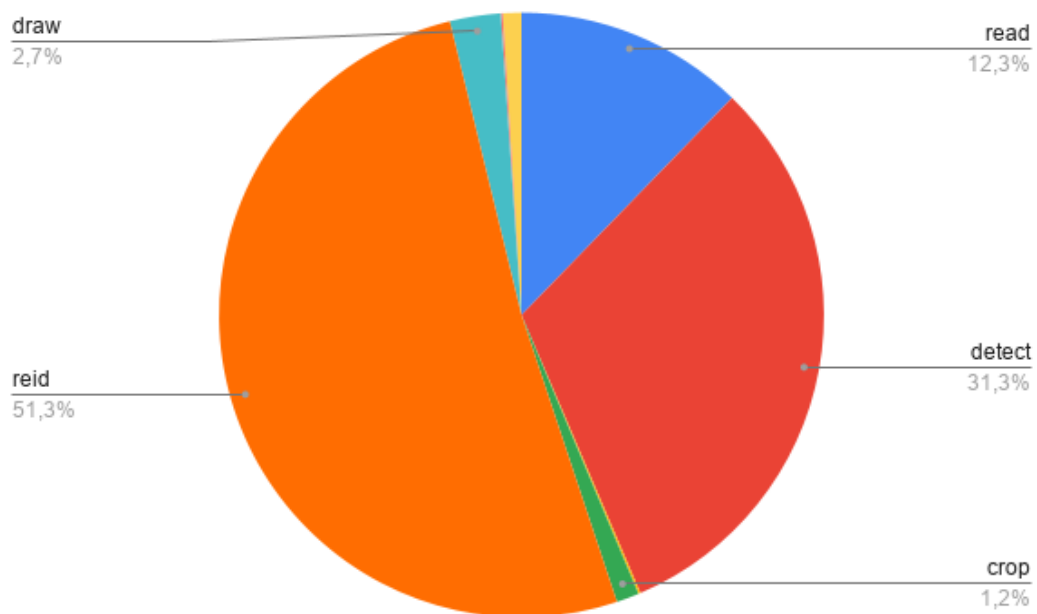


Рисунок 13 – Диаграмма долей временных затрат

Таким образом, итоговая экспериментальная оценка показывает, что для последовательности MOT17-11-SDP разработанный модуль способен поддерживать высокую точность наблюдений, высокую чистоту глобальных идентификаторов и, главное, устойчиво повторно назначать один и тот же глобальный идентификатор после разрыва наблюдения. Ограничением системы остается скорость работы: в исследованной конфигурации модуль пока не достигает режима реального времени для потока 30 кадр/с, поэтому дальнейшее развитие прикладной версии должно быть связано прежде всего с ускорением этапов детекции и извлечения признаков.

4.6 Сравнение решения с аналогами

Сравнение реализованного программного модуля представлено в таблице 8. Обозначения критериев:

1. Готовый видеоконвейер – решение можно использовать как законченную систему обработки видеопотока, а не только как отдельный алгоритм или библиотеку.
2. Галерея идентичностей – идентичности хранятся в самостоятельной структуре, не сводятся только к локальному трекеру или его буферу.
3. Межсеансовое сохранение – состояние идентичностей может сохраняться между запусками системы.
4. Конфигурационное управление – параметры конвейера можно изменять через внешний конфигурационный профиль без переработки кода.
5. Отсутствие привязки к экосистеме – решение не требует жёсткой зависимости от специфической платформы, SDK или среды исполнения.
6. Готово без дополнительного кода – решение можно использовать по назначению без необходимости отдельно дописывать обёртку, недостающие подсистемы или собственную логику ReID-галереи.

Обозначения в таблице:

- + – свойство явно присутствует в решении или прямо заявлено в его описании в тексте ВКР.
- - – свойство не заявлено, отсутствует или для практического использования требуется дополнительная реализация.

Таблица 8 – Сравнение результата с аналогами

Критерий	1	2	3	4	5	6
Реализованный программный модуль	+	+	+	+	+	+
Deep SORT	-	-	-	-	+	-
ByteTrack	-	-	-	-	+	-
BoT-SORT	-	-	-	-	+	-
Ultralytics Tracking	-	-	-	+	+	-
NVIDIA DeepStream	+	+	-	+	-	-
OpenVINO ReID / deep-object-reid	+	+	-	+	-	-
Torchreid	-	-	-	-	+	-
FastReID	-	-	-	-	+	-
Open-ReID	-	-	-	-	+	-

Сравнение с рассмотренными в работе аналогами показывает, что существующие решения закрывают разные части задачи повторной идентификации, но не совпадают с постановкой настоящего исследования полностью. Разработанный модуль сочетает готовый видеоконвейер, отделённую галерею идентичностей, сохранение состояния между сеансами, конфигурационное управление параметрами, низкую привязку к внешней экосистеме и готовность к использованию без дописывания дополнительной обёртки системы ReID. Тем самым подтверждается, что цель работы достигнута.

4.7 Выводы по результатам исследования

В ходе исследования были определены экспериментальные условия проверки разработанного программного модуля, выбраны тестовые видеофрагменты и зафиксированы параметры вычислительной системы. Для оценки использовались последовательности MOT17 и дополнительный

тестовый видеофрагмент, что позволило проверить работу модуля как в условиях плотной пешеходной сцены, так и при обработке нескольких классов объектов.

Функциональная проверка показала, что разработанный модуль выполняет основные требования, сформулированные к системе. Была подтверждена возможность сохранения и повторного использования галереи идентичностей между независимыми запусками, работа с несколькими классами объектов, а также загрузка пользовательских весов модели извлечения признаков без изменения общей логики конвейера.

Для количественной оценки были использованы метрики качества детекции, сопоставления идентичностей и производительности. Такой набор показателей позволил оценить точность обнаружения объектов и основное свойство разрабатываемой системы – устойчивость повторного назначения глобального идентификатора после временной потери объекта.

Параметрический анализ показал, что результат работы системы определяется не одним отдельным параметром, а совместным влиянием порога детекции, порогов сопоставления с галереей, коэффициента обновления прототипа и параметров локального трекинга. Итоговая экспериментальная оценка показала, что разработанный модуль способен поддерживать согласованную структуру глобальных идентичностей и повторно назначать объекту ранее созданный идентификатор после его исчезновения из кадра.

Сравнение с аналогами показало, что разработанное решение сочетает готовый видеоконвейер, самостоятельную галерею идентичностей, межсеансовое сохранение состояния, конфигурационное управление и отсутствие жесткой привязки к конкретной программно-аппаратной экосистеме.

5 Экономическое обоснование проекта

Цель выпускной квалификационной работы – разработка программного модуля повторной идентификации объектов для применения в системах видеонаблюдения и видеоаналитики.

5.1 Календарный план выполнения

Определим затраты на этапе проектирования. Для этого определим продолжительность каждой работы по следующей формуле (19):

$$t_j^0 = \frac{3t_{min} + 2t_{max}}{5}, \quad (19)$$

где t_j^0 – ожидаемая длительность -й работы; t_{min} и t_{max} – наименьшая и наибольшая по мнению эксперта длительность работы (в часах) (см. таблица 9).

Таблица 9 – Длительность этапа разработки

№	Наименование работы	Длительность работы, ч		
		t_{min}	t_{max}	t_0
1	Разработка задания на ВКР	2	5	3,2
2	Обзор литературы по теме работы	15	22	17,8
3	Обзор предметной области и существующих аналогов	20	28	23,2
4	Формулировка требований к решению и постановка задачи	10	15	12,0
5	Описание метода решения и проектирование архитектуры	17	24	19,8
6	Реализация подсистемы детекции и локального трекинга	27	39	31,8
7	Реализация подсистемы извлечения признаков и галереи идентичностей	32	44	36,8
8	Реализация конфигурационного управления, журналирования и сохранения результатов	10	15	12,0

Продолжение таблицы 9

№	Наименование работы	Длительность работы, ч		
		t _{min}	t _{max}	t ₀
9	Исследование свойств разработанного решения	24	35	28,4
10	Экономическое обоснование проекта	10	15	12,0
11	Оформление пояснительной записки	24	32	27,2
12	Оформление иллюстративного материала	7	10	8,2
13	Проверка ВКР	2	3	2,4
14	Подготовка к защите	7	10	8,2
Итого		207	297	243,0

5.2 Расходы на оплату труда

5.2.1 Основная заработная плата

Для расчета заработной платы исполнителей по времени необходимо установить определенную ставку. Эта ставка рассчитывается на основе месячного дохода исполнителя. Дневная ставка определяется делением месячного оклада на стандартное количество рабочих дней (обычно 21 день). Часовая ставка получается разделением месячной заработной платы на общее количество рабочих часов в месяце, которое обычно равно 168 часам (21 день, умноженный на 8 часов рабочего дня).

Средняя заработная плата руководителя доцента составляет 97 000 руб., средняя заработная плата инженера на 2026 год составляет 84 000 руб. [45,46]

Часовая ставка студента равна $\frac{84000}{168} = 500$ руб./час.

Часовая ставка руководителя равна $\frac{97000}{168} = 577,38$ руб./час.

Рассчитаем расходы на оплату труда задействованных лиц (см. таблицу 10)

Таблица 10 – Расчет ставки исполнителей

№	Этапы и содержание выполняемых работ	Исполнитель	Трудоемкость, ед.ч	Ставка, руб./час
1	Разработка задания на ВКР	Руководитель	3,2	577,38
2	Обзор литературы по теме работы	Студент	17,8	500
3	Обзор предметной области и существующих аналогов	Студент	23,2	500
4	Формулировка требований к решению и постановка задачи	Студент	12,0	500
5	Описание метода решения и проектирование архитектуры	Студент	19,8	500
6	Реализация подсистемы детекции и локального трекинга	Студент	31,8	500
7	Реализация подсистемы извлечения признаков и галереи идентичностей	Студент	36,8	500
8	Реализация конфигурационного управления, журналирования и сохранения результатов	Студент	12,0	500
9	Исследование свойств разработанного решения	Студент	28,4	500
10	Экономическое обоснование проекта	Студент	12,0	500
11	Оформление пояснительной записки	Студент	27,2	500
12	Оформление иллюстративного материала	Студент	8,2	500
13	Проверка ВКР	Руководитель	2,4	577,38

Продолжение таблицы 10

№	Этапы и содержание выполняемых работ	Исполнитель	Трудоемкость, ед.ч	Ставка, руб./час
14	Подготовка к защите	Студент	8,2	500

На основе полученных данных о трудоемкости выполняемых работ и ставке за час определяются расходы на заработную плату каждого исполнителя и отчисления на страховые взносы на обязательное социальное, пенсионное и медицинское страхование.

Расходы на основную заработную плату исполнителей определяются по следующей формуле:

$$З_{\text{осн.з/пл}} = \sum_{i=1}^k T_i * C_i,$$

где $З_{\text{осн.з/пл}}$ – расходы на основную заработную плату исполнителей (руб.); k – количество исполнителей; T_i – время, затраченное i -м исполнителем на проведение исследования (час.); C_i – ставка i -го исполнителя (руб./час).

Основная заработная плата студента равна 118 700 руб.

Основная заработная плата руководителя равна 3 233,33 руб.

Суммируя, получаем, что затраты на основную заработную плату составят 121 933,33 руб.

5.2.2 Дополнительная заработная плата

Расходы на дополнительную заработную плату исполнителей определяются по следующей формуле:

$$З_{\text{доп.з/пл}} = З_{\text{осн.з/пл}} * \frac{H_{\text{доп}}}{100},$$

где $З_{\text{доп.з/пл}}$ – расходы на дополнительную заработную плату исполнителей, $З_{\text{осн.з/пл}}$ – расходы на основную заработную плату исполнителей, $H_{\text{доп}}$ – норматив дополнительной заработной платы, %. При выполнении расчетов в ВКР норматив дополнительной заработной платы принимается равным 14% [47].

$$З_{\text{доп.з./пл}} = 121\,933,33 * \frac{14}{100} = 17\,070,67 \text{ руб.}$$

5.2.3 Отчисления на страховые взносы

Страховые взносы по статье «Отчисления на социальные нужды» отчисляются в следующие государственные внебюджетные фонды социального назначения:

- Социальный фонд России (СФР);
- Федеральный фонд обязательного медицинского страхования (ФФОМС).

Отчисления на страховые взносы на обязательное социальное, пенсионное и медицинское страхование с основной и дополнительной заработной платы исполнителей определяются по следующей формуле (22):

$$З_{\text{соц}} = (З_{\text{осн.з./пл}} + З_{\text{доп.з./пл}}) * \frac{Н_{\text{соц}}}{100}, \quad (22)$$

где $З_{\text{соц}}$ – отчисления на социальные нужды с заработной платы, руб.;
 $З_{\text{осн.з./пл}}$ – расходы на основную заработную плату исполнителей, руб.;
 $З_{\text{доп.з./пл}}$ – расходы на дополнительную заработную плату исполнителей, руб.;
 $Н_{\text{ст}}$ – норматив отчислений страховых взносов на обязательное социальное, пенсионное и медицинское страхование, согласно Налоговому кодексу РФ составляет 30% [48].

$$З_{\text{соц}} = (121\,933,33 + 17\,070,67) * \frac{30}{100} = 41\,701,20 \text{ руб.}$$

5.3 Материальные расходы

Расчет затрат на материалы производится по следующей формуле (23):

$$З_{\text{м}} = \sum_{l=1}^L G_l Ц_l \left(1 + \frac{Н_{\text{т.з.}}}{100}\right), \quad (23)$$

где $З_{\text{м}}$ – затраты на сырье и материалы (руб.); l – индекс сырья или материала; G_l – норма расхода l -го материала на единицу продукции (ед.); $Ц_l$ – цена приобретения единицы l -го материала (руб./ед.); $Н_{\text{т.з.}}$ – норма транспортно-заготовительных расходов (%).

Норму транспортно-заготовительных расходов принимаем равной 10% [47].

Результаты расчетов представлены в таблице 11

Таблица 11 – Затраты на сырье и материалы

Изделие	Материал	Тип	Норма расхода, ед.	Цена за единицу, руб.	Сумма на изделие, руб.
Ручка шариковая	Пластик, чернила, резина	Ручка шариковая автоматическая BIC, синяя, 1 мм	1	79	79
Бумага офисная	Бумага	"SvetoCopy", 500 листов, А4.	1	399	399
Картридж для принтера	Тонер, пластик, металл	Картридж ТК-1150 черный	1	805	805

$$З_{\text{м}} = (79 * 1 + 399 * 1 + 805 * 1) * \left(1 + \frac{10}{100}\right) = 1411,3 \text{ руб.}$$

5.4 Нематериальные расходы

Нематериальные расходы подразумевают под собой расходы на программное обеспечение. При выполнении работы использовалось только свободное программное обеспечение, поэтому данная статья расходов отсутствует.

5.5 Амортизационные отчисления

Амортизационные отчисления по основному средству и за год определяются по следующей формуле (24):

$$A_i = Ц_{\text{п.н.}i} * \frac{H_{ai}}{100}, \quad (24)$$

где A_i – амортизационные отчисления за год по i -му основному средству, руб.; $Ц_{\text{п.н.}i}$ – первоначальная стоимость i -го основного средства, руб.; H_{ai} – годовая норма амортизации i -го основного средства, %.

Основным средством, используемым в ходе написания работы, будем считать личный ноутбук стоимостью 70000 руб. Использовался он в течение трех месяцев, а срок службы составляет 4 года [49].

Для начисления амортизации выберем линейный способ. Тогда она составит 25% в год (17 500 руб.)

Величина амортизационных отчислений по i -му основному средству, используемому студентом при работе над БКР, определяется по формуле (25):

$$A_{i\text{БКР}} = A_i * \frac{T_{i\text{БКР}}}{12}, \quad (25)$$

где $A_{i\text{БКР}}$ – амортизационные отчисления по i -му основному средству, используемому студентом в работе над БКР, руб.; A_i – амортизационные отчисления за год по i -му основному средству, руб.; $T_{i\text{БКР}}$ – время, в течение которого студент использует i -е основное средство, мес.

$$A_{1\text{БКР}} = 17\,500 * \frac{3}{12} = 4\,375 \text{ руб.}$$

5.6 Затраты на содержание и эксплуатацию

Затраты на содержание и эксплуатацию оборудования определяются из расчета на 1 час работы оборудования с учетом стоимости и производительности оборудования рассчитываются по формуле (26):

$$З_{эо} = \sum_{i=1}^m C_i^{\text{мч}} * t_i^{\text{м}}, \quad (26)$$

где $З_{эо}$ – затраты на содержание и эксплуатацию оборудования (руб.); $C_i^{\text{мч}}$ – расчетная себестоимость одного машино-часа работы оборудования на i -й технологической операции (руб./м-ч); $t_i^{\text{м}}$ – количество машино-часов, затрачиваемых на выполнение i -й технологической операции (м-ч).

Мощность ноутбука составляет порядка 180 Вт, величина тарифа составляет 7,08 руб./кВт*ч.

$$C^{\text{мч}} = 0,18 * 7,08 = 1,27 \text{ руб./м-ч.}$$

$$З_{эо} = 1,27 * 243 = 308,61 \text{ руб.}$$

5.7 Услуги сторонних организаций

К этому пункту отнесем услуги по предоставлению доступа в интернет. Цена услуги составляет 600 руб./мес. [50], предоставление услуги нужно на протяжении 3-х месяцев, а значит, стоимость составит:

$$З_y = 600 * 3 = 1\,800 \text{ руб.}$$

5.8 Накладные расходы

Накладные расходы составляют 20% от суммы основной и дополнительной заработной платы и равны 27 800,79 руб.

Все проведённые расчеты сведем в таблицу 12.

Таблица 12 – Итоговые затраты

№	Наименование статьи расходов	Стоимость, руб.	Удельный вес статьи затрат, %
1	Оплата труда	139 004,00	64,23
2	Отчисления на страховые взносы	41 701,20	19,27
3	Сырье и материалы	1 411,30	0,65
5	Амортизационные отчисления	4 375,00	2,02
6	Затраты на содержание и эксплуатацию	308,61	0,14
7	Услуги сторонних организаций	1 800	0,83
8	Накладные расходы	27 800,80	12,85
Итого:		217 847,34	100

5.9 Вывод

Разработка программного модуля повторной идентификации объектов в видеопотоках камер видеонаблюдения позволяет автоматизировать часть задач видеоаналитики, связанных с долговременным сопоставлением объектов после разрыва наблюдения. Использование такого решения снижает зависимость от ручного анализа видеоданных, повышает устойчивость

идентификации и упрощает обработку видеопотока в прикладных системах наблюдения. Основная часть затрат при выполнении работы связана с оплатой труда разработчика и использованием вычислительной техники. Вложения в создание такого модуля являются оправданными, поскольку результат работы может использоваться как основа для дальнейшего развития и внедрения интеллектуальных систем видеонаблюдения.

ЗАКЛЮЧЕНИЕ

В результате выполнения ВКР был получен программный модуль повторной идентификации объектов в видеопотоке с камер видеонаблюдения. Модуль выполняет обнаружение объектов, локальное сопровождение, извлечение признаков представлений, сопоставление с галереей идентичностей и повторное назначение глобального идентификатора после разрыва наблюдения. Следовательно, цель работы достигнута.

В ходе решения первой задачи был выполнен анализ предметной области повторной идентификации объектов. Рассмотрены отличия повторной идентификации от обнаружения объектов и локального трекинга, а также проанализированы краткосрочные и долгосрочные архитектуры ReID-систем. Отдельно были рассмотрены существующие аналоги: трекеры Deep SORT, ByteTrack, BoT-SORT, инструменты Ultralytics, NVIDIA DeepStream, OpenVINO, Torchreid и FastReID. По результатам сравнения установлено, что готовые трекеры в основном поддерживают идентичность в пределах локальной траектории или буфера трекера, а библиотеки ReID не являются законченными видеоконвейерами.

В ходе решения второй задачи были сформулированы требования к программному модулю. В требования включены поддержка видеофайлов и потоковых источников, обнаружение объектов, локальное сопровождение, извлечение признаков, создание и повторное назначение глобальных идентификаторов, сохранение состояния галереи, ограничение ее размера, конфигурационное управление параметрами и формирование структурированных результатов работы.

В ходе решения третьей задачи была спроектирована модульная архитектура системы. В ней выделены подсистемы получения кадров, обнаружения объектов, локального трекинга, извлечения признаков, сопоставления с галереей, обновления галереи и сохранения результатов.

Такое разделение позволило отделить краткосрочную локальную траекторию от глобальной идентичности объекта.

В ходе решения четвертой задачи был реализован программный модуль на языке Python. Для обработки видеоданных использована библиотека OpenCV, для обнаружения объектов – модель семейства YOLO, для извлечения признаков – модели библиотеки Torchreid. Реализованы конфигурационное управление, сохранение результатов обработки, сохранение состояния галереи и сценарии запуска через командный интерфейс и Python-библиотеку.

В ходе решения пятой задачи была проведена экспериментальная проверка свойств разработанного модуля. На видеопоследовательностях MOT17 и тестовых видеофрагментах исследовались работоспособность функциональных возможностей, влияние параметров детекции и сопоставления, качество повторного связывания после разрыва наблюдения и производительность. По результатам экспериментов выбрана зафиксирована конфигурация для исследования точности и производительности системы.

В ходе решения шестой задачи было выполнено экономическое обоснование проекта. Рассчитаны затраты на оплату труда, страховые взносы, материальные и нематериальные расходы, амортизационные отчисления, затраты на содержание и эксплуатацию, услуги сторонних организаций и накладные расходы. Итоговая стоимость разработки составила 217 847,34 руб.

Дальнейшее развитие работы может быть связано со следующими направлениями: автоматический подбор параметров конфигурации на основе предоставленного видеофрагмента, расширение системы для работы с несколькими камерами, оптимизация производительности за счет экспорта моделей в ONNX и применения аппаратного ускорения, развитие галереи идентичностей за счет более сложных стратегий обновления прототипов, учета времени отсутствия объекта и защиты от накопления ошибочных признаков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Репозиторий проекта [Электронный ресурс] // GitHub. URL: <https://github.com/CactysFedya/re-id> (дата обращения: 12.05.2026).
2. MarketsandMarkets. Video Analytics Market – Global Forecast to 2028 [Электронный ресурс]. URL: <https://www.marketsandmarkets.com/Market-Reports/intelligent-video-analytics-market-778.html> (дата обращения: 06.04.2026).
3. Techportal. Рынок видеоаналитики в цифрах [Электронный ресурс]. URL: <https://www.techportal.ru/security/video-analytics/rynok-v-tsifrakh/> (дата обращения: 07.04.2026).
4. Ye M. и др. Deep Learning for Person Re-identification: A Survey and Outlook: arXiv:2001.04193. arXiv, 2021.
5. Chen Kh. и др. Povtornaya identifikatsiya lyudey v sistemakh videonablyudeniya s ispol'zovaniem glubokogo obucheniya: analiz sushchestvuyushchikh metodov // Avtomatika i telemekhanika. 2023. № 5. С. 61–112.
6. Martini M., Paolanti M., Frontoni E. Open-World Person Re-Identification With RGBD Camera in Top-View Configuration for Retail Applications // IEEE Access. 2020. Т. 8. С. 67756–67765.
7. Tang Z. и др. CityFlow: A City-Scale Benchmark for Multi-Target Multi-Camera Vehicle Tracking and Re-Identification: arXiv:1903.09254. arXiv, 2019.
8. Ihnatsyeva S. A., Bohush R. P. Person re-identification algorithm by image from video surveillance sistem using a neural network compound descriptor // Sist. anal.i prikl. inform. 2024. № 1. С. 12–17.
9. Wojke N., Bewley A., Paulus D. Simple Online and Realtime Tracking with a Deep Association Metric: arXiv:1703.07402. arXiv, 2017.
10. Zhang Y. и др. ByteTrack: Multi-Object Tracking by Associating Every Detection Box: arXiv:2110.06864. arXiv, 2022.

11. Uddin M. K. и др. Person Re-Identification with RGB–D and RGB–IR Sensors: A Comprehensive Survey // Sensors. 2023. Т. 23, № 3. С. 1504.
12. Wu A. и др. RGB-Infrared Cross-Modality Person Re-identification // 2017 IEEE International Conference on Computer Vision (ICCV). Venice: IEEE, 2017. С. 5390–5399.
13. Aharon N., Orfaig R., Bobrovsky B.-Z. BoT-SORT: Robust Associations Multi-Pedestrian Tracking: arXiv:2206.14651. arXiv, 2022.
14. Ultralytics YOLO Docs. Multi-Object Tracking with Ultralytics YOLO [Электронный ресурс]. URL: <https://docs.ultralytics.com/modes/track/> (дата обращения: 11.04.2026).
15. Zhou K. и др. Omni-Scale Feature Learning for Person Re-Identification. arXiv, 2019.
16. He S. и др. TransReID: Transformer-based Object Re-Identification. arXiv, 2021.
17. Bi Y. и др. Enhancing Person Re-Identification through Attention-Driven Global Features and Angular Loss Optimization // Entropy. 2024. Т. 26, № 6. С. 436.
18. NVIDIA DeepStream Documentation. Gst-nvtracker [Электронный ресурс]. URL: https://docs.nvidia.com/metropolis/deepstream/8.0/text/DS_plugin_gst-nvtracker.html (дата обращения: 13.04.2026).
19. OpenVINO Documentation. Multi Camera Multi Target Python Demo [Электронный ресурс]. URL: https://docs.openvino.ai/2023.3/omz_demos_multi_camera_multi_target_tracking_demo_python.html (дата обращения: 14.04.2026).
20. OpenVINO Documentation. person-reidentification-retail-0286 [Электронный ресурс]. URL: https://docs.openvino.ai/2023.3/omz_models_model_person_reidentification_retail_0286.html (дата обращения: 15.04.2026).
21. Wang S. и др. The 8th AI City Challenge. arXiv, 2024.

22. Zhou K., Xiang T. Torchreid: A Library for Deep Learning Person Re-Identification in Pytorch: arXiv:1910.10093. arXiv, 2019.
23. He L. и др. FastReID: A Pytorch Toolbox for General Instance Re-identification: arXiv:2006.02631. arXiv, 2020.
24. Open-ReID documentation [Электронный ресурс]. URL: <https://cysu.github.io/open-reid/>. (дата обращения: 16.04.2026).
25. Ali M. L., Zhang Z. The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection // Computers. 2024. Т. 13, № 12. С. 336.
26. Python Software Foundation. tomlib — Parse TOML files [Электронный ресурс]. URL: <https://docs.python.org/3/library/tomllib.html> (дата обращения: 22.04.2026).
27. Python Software Foundation. Python 3 Documentation [Электронный ресурс]. URL: <https://docs.python.org/3/> (дата обращения: 22.04.2026).
28. OpenCV. cv::VideoWriter Class Reference [Электронный ресурс]. URL: https://docs.opencv.org/4.x/dd/d9e/classcv_1_1VideoWriter.html (дата обращения: 23.04.2026).
29. OpenCV. cv::VideoCapture Class Reference [Электронный ресурс]. URL: https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html (дата обращения: 23.04.2026).
30. Ultralytics YOLO Docs. Predict Mode [Электронный ресурс]. URL: <https://docs.ultralytics.com/modes/predict/> (дата обращения: 24.04.2026).
31. Lin T.-Y. и др. Microsoft COCO: Common Objects in Context. arXiv, 2014.
32. Ristani E. и др. Performance Measures and a Data Set for Multi-target, Multi-camera Tracking // Computer Vision – ECCV 2016 Workshops / под ред. Hua G., Jégou H. Cham: Springer International Publishing, 2016. Т. 9914. С. 17–35.

33. Luiten J. и др. HOTA: A Higher Order Metric for Evaluating Multi-object Tracking // Int J Comput Vis. 2021. Т. 129, № 2. С. 548–578.
34. ONNX Documentation. Overview [Электронный ресурс]. URL: <https://onnx.ai/onnx/repo-docs/Overview.html> (дата обращения: 08.05.2026).
35. ONNX Runtime Documentation. Performance [Электронный ресурс]. URL: <https://onnxruntime.ai/docs/performance/> (дата обращения: 09.05.2026).
36. SQLite Documentation. Backup API [Электронный ресурс]. URL: <https://sqlite.org/backup.html> (дата обращения: 10.05.2026).
37. Dendorfer P. и др. MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking // Int J Comput Vis. 2021. Т. 129, № 4. С. 845–881.
38. Zheng L. и др. Scalable Person Re-identification: A Benchmark // 2015 IEEE International Conference on Computer Vision (ICCV). Santiago, Chile: IEEE, 2015. С. 1116–1124.
39. Bernardin K., Stiefelhausen R. Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics // EURASIP Journal on Image and Video Processing. 2008. Т. 2008. С. 1–10.
40. Amigó E. и др. A comparison of extrinsic clustering evaluation metrics based on formal constraints // Inf Retrieval. 2009. Т. 12, № 4. С. 461–486.
41. Zhou K. Torchreid 1.4.0 Documentation. User Guide and Model Zoo [Электронный ресурс]. URL: <https://kaiyangzhou.github.io/deep-person-reid/> (дата обращения: 25.04.2026).
42. He K. и др. Deep Residual Learning for Image Recognition. arXiv, 2015.
43. Sandler M. и др. MobileNetV2: Inverted Residuals and Linear Bottlenecks. arXiv, 2018.
44. Zhong Z. и др. Generalizing a Person Retrieval Model Hetero- and Homogeneously // Computer Vision – ECCV 2018 / под ред. Ferrari V. и др. Cham: Springer International Publishing, 2018. Т. 11217. С. 176–192.

45. Средняя заработная плата по должности доцент в 2026 [Электронный ресурс]. URL: <https://dreamjob.ru/salary/docent> (дата обращения: 07.05.2026).

46. Средняя заработная плата по должности инженер в 2026 [Электронный ресурс]. URL: <https://dreamjob.ru/salary/inzhener> (дата обращения: 07.05.2026).

47. О. Г. Алексеева. Учебно-методическое пособие по выполнению дополнительного раздела «Экономическое обоснование ВКР». Санкт-Петербург: Издательство СПбГЭТУ «ЛЭТИ» 2023.

48. Статья 425 НК РФ [Электронный ресурс] // КонсультантПлюс. URL: https://www.consultant.ru/document/cons_doc_LAW_28165/a3f603ffd57b1431ed51e1693ba710093347235d (дата обращения: 08.05.2026).

49. Постановление Правительства РФ от 01 января 2002 г. № 1 «О классификации основных средств, включаемых в амортизационные группы» (ред. от 18.11.2022 г.) [Электронный ресурс] // КонсультантПлюс. URL: https://www.consultant.ru/document/cons_doc_LAW_34710 (дата обращения: 09.05.2026).

50. Тарифы на домашний интернет [Электронный ресурс] // Ростелеком. URL: <https://spb.rt.ru/homeinternet> (дата обращения: 10.05.2026).